



APPLIED SOFTWARE ENGINEERING SERIES

Requirements Engineering for Software and Systems

FOURTH EDITION

Phillip A. Laplante
Mohamad H. Kassab



CRC Press
Taylor & Francis Group

AN AUERBACH BOOK

Requirements Engineering for Software and Systems



Taylor & Francis

Taylor & Francis Group

<http://taylorandfrancis.com>

Requirements Engineering for Software and Systems

Fourth Edition

Phillip A. Laplante
Mohamad H. Kassab

Fourth Edition published 2022
by CRC Press
6000 Broken Sound Parkway NW, Suite 300, Boca Raton, FL 33487-2742

and by CRC Press
4 Park Square, Milton Park, Abingdon, Oxon, OX14 4RN

First edition published by CRC Press 2009

CRC Press is an imprint of Taylor & Francis Group, LLC

© 2022 Phillip A. Laplante and Mohamad H. Kassab

Reasonable efforts have been made to publish reliable data and information, but the author and publisher cannot assume responsibility for the validity of all materials or the consequences of their use. The authors and publishers have attempted to trace the copyright holders of all material reproduced in this publication and apologize to copyright holders if permission to publish in this form has not been obtained. If any copyright material has not been acknowledged please write and let us know so we may rectify in any future reprint.

Except as permitted under U.S. Copyright Law, no part of this book may be reprinted, reproduced, transmitted, or utilized in any form by any electronic, mechanical, or other means, now known or hereafter invented, including photocopying, microfilming, and recording, or in any information storage or retrieval system, without written permission from the publishers.

For permission to photocopy or use material electronically from this work, access www.copyright.com or contact the Copyright Clearance Center, Inc. (CCC), 222 Rosewood Drive, Danvers, MA 01923, 978-750-8400. For works that are not available on CCC please contact mpkbookspermissions@tandf.co.uk

Trademark notice: Product or corporate names may be trademarks or registered trademarks and are used only for identification and explanation without intent to infringe.

Library of Congress Cataloging-in-Publication Data

Names: Laplante, Phillip A., author. | Kassab, Mohamad, author.

Title: Requirements engineering for software and systems / Phillip A.

Laplante and Mohamad H. Kassab.

Description: Boca Raton, FL : CRC Press, 2022. | Series: Applied software engineering series | Includes bibliographical references and index.

Identifiers: LCCN 2022000663 | ISBN 9780367654528 (hbk) |

ISBN 9781032275994 (pbk) | ISBN 9781003129509 (ebk)

Subjects: LCSH: Computer programs—Specifications. |

Computer software—Development. | Software engineering. | Requirements engineering.

Classification: LCC QA76.76.S73 L37 2022 | DDC 005.1068—dc23/eng/20220322

LC record available at <https://lccn.loc.gov/2022000663>

ISBN: 978-0-367-65452-8 (hbk)

ISBN: 978-1-032-27599-4 (pbk)

ISBN: 978-1-003-12950-9 (ebk)

DOI: 10.1201/9781003129509

Typeset in Garamond
by codeMantra

Contents

Preface	xv
Acknowledgments	xxi
Authors	xxiii
1 Introduction to Requirements Engineering	1
Motivation	1
What Is Requirements Engineering?	1
You Probably Don't Do Enough Requirements Engineering	2
What Are Requirements?	3
Requirements vs. Features vs. Goals	3
Requirements Classifications	4
Requirements Level Classification	4
Requirements Specifications Types	5
Functional Requirements	6
Nonfunctional Requirements	7
Domain Requirements	7
Domain Vocabulary Understanding	7
Requirements Engineering Activities	9
Requirements Elicitation/Discovery	9
Requirements Analysis and Agreement	9
Requirements Representation	9
Requirements Validation	10
Requirements Management	10
Bodies of Knowledge	10
The Requirements Engineer	13
Requirements Engineer Roles	14
Requirements Engineer as Software or Systems Engineer	14
Requirements Engineer as Subject Matter Expert	14
Requirements Engineer as Architect	15
Requirements Engineer as Business Process Expert	15
Ignorance as Virtue	15
Role of the Customer	16
Problems with Traditional Requirements Engineering	16
Complexity	17
Gold-Plating and Ridiculous Requirements	18
Obsolete Requirements	18

Four Dark Corners	19
Difficulties in Enveloping System Behavior	20
The Danger of “All” in Specifications	21
Exercises	22
References	23
2 Preparing for Requirements Elicitation	27
Product Business Goals and Mission Statements	27
Encounter with a Customer	29
Identifying the System Boundaries	30
Context Diagrams	30
Stakeholders	31
Negative Stakeholders	32
Stakeholder Identification	32
Stakeholder Identification Questions	32
Rich Pictures	34
Stakeholder/User Classes	36
Stakeholders vs. Use Case Actors	37
User Characteristics	37
Customer Wants and Needs	38
What Do Customers Want?	38
What Don't Customers Want?	40
Why Do Customers Change Their Minds?	41
Stakeholder Prioritization	42
Communicating with Customers and Other Stakeholders	43
Managing Expectations	44
Stakeholder Negotiations	48
Uncovering Stakeholder Goals	49
Exercises	51
Note	52
References	52
3 Requirements Elicitation	55
Introduction	55
Requirements Elicitation - First Step	56
Elicitation Techniques Survey	57
Brainstorming	58
Card Sorting	58
Crowdsourcing	59
Designer as Apprentice	61
Domain Analysis	62
Ethnographic Observation	62
Goal-Based Approaches	63
Group Work	64
Interviews	65
Introspection	66
Joint Application Design	67

Laddering	68
Protocol Analysis.....	68
Prototyping.....	69
Quality Function Deployment.....	71
Questionnaires/Surveys.....	73
Repertory Grids	74
Reverse Engineering.....	74
Scenarios.....	75
Task Analysis.....	75
Use Cases.....	76
User Stories.....	78
Viewpoints.....	78
Workshops.....	79
Eliciting Nonfunctional Requirements	79
Elicitation Summary	80
Which Combination of Requirements Elicitation Techniques Should Be Used?.....	80
Prevalence of Requirements Elicitation Techniques	84
Eliciting Hazards.....	84
Misuse Cases.....	85
Antimodels	85
Formal Methods	86
Exercises.....	88
Note	88
References	89
4 Writing the Requirements Document	91
Requirements Agreement and Analysis.....	91
Requirements Representation.....	92
Approaches to Requirements Representation	92
ISO/IEC/IEEE Standard 29148.....	94
Recommendations on Representing Nonfunctional Requirements.....	95
Recommendations on Representing Functional Requirements	96
Operating System.....	97
Command Validation	98
UML/SysML	99
The Requirements Document	100
Users of a Requirements Document.....	100
Requirements Document Requirements.....	101
Preferred Writing Style	101
Text Structure Analysis.....	102
Requirement Format	102
Use of Imperatives.....	104
Shall or Shall Not?.....	104
Avoiding Imprecision in Requirements.....	105
Requirements Document Size	107
Behavioral Specifications	107
Best Practices and Recommendations.....	108

Exercises.....	110
References	111
5 On Nonfunctional Requirements.....	113
Motivation to Consider NFRs Earlier in Development	113
What Is an NFR?	115
What Are the Different Types of NFRs?	117
Quality Requirements in Practice.....	119
Specifying with Quality Attribute Scenarios	121
Specifying with Planguage.....	121
Popular Quality Requirements	122
Security.....	122
Modifiability.....	123
Interoperability	124
Performance.....	124
Testability	124
Usability	124
Other Approaches to Handle NFRs in RE	125
NFR Framework.....	125
Incorporating NFRs with UML Models.....	126
Exercises.....	128
References	128
6 Requirements Validations and Verifications.....	131
What Is Requirements Risk Management?	131
Validation and Verification	134
Techniques for Requirements V&V	135
Walkthroughs.....	135
Inspections	135
Goal-Based Requirements Analysis.....	136
Requirements Understanding	137
Validating Requirements Use Cases	138
Prototyping	138
Tools for V&V	138
Requirements V&V Matrices.....	140
Standards for V&V	141
ISO/IECIEEE Standard 29148.....	142
Singularity.....	143
Feasibility	144
Ambiguity	144
Completeness.....	145
Consistency.....	145
Verifiability.....	147
Traceability.....	147
Ranking	148
Example Validation of Requirements	148
Singular.....	148

Feasible	149
Unambiguous.....	149
Complete	149
Consistent.....	149
Verifiable.....	150
Traceable	150
Ranked.....	150
The Importance of Measurement in Requirements V&V	150
Goal/Question/Metric Analysis	151
NASA Requirements Testing.....	153
NASA ARM Tool	154
Imperatives.....	154
Continuances	156
Directives	156
Options.....	156
Weak Phrases.....	156
Incomplete	157
Subjects.....	158
Specification Depth.....	158
Readability Statistics.....	160
Summary of NASA Metrics.....	161
Exercises.....	164
Note.....	164
References	165
7 Formal Methods	167
Motivation	167
What Are Formal Methods?	168
Formal Methods Classification.....	169
A Little History.....	169
Using Formal Methods	170
Examples.....	170
Formalization of Train Station in B	171
Formalization of Space Shuttle Flight Software Using Mur Φ	173
Formalization of an Energy Management System Using Category Theory	174
Example: An Energy Management System	176
Other Notable Formal Methods for Requirements Engineering.....	178
Requirements Validation.....	178
Consistency Checking Using Truth Tables.....	179
Consistency Checking by Inspection	180
Consistency Checking Limitations	180
Theorem Proving	182
Program Correctness	182
Hoare Logic	182
Model Checking	185
Integrated Tools	186

Which Formal Approach to Use?	187
Objections, Myths, and Limitations	187
Objections and Myths.....	188
Limitations of Formal Methods	189
Combining Formal and Informal Methods	190
Bowen and Hinchey's Advice	190
Exercises.....	192
Notes	193
References	193
8 Requirements Specification and Agile Methodologies	195
Introduction to Agile Methodologies.....	195
Principles Behind the Agile Manifesto.....	196
Benefits of Agile Software Development.....	197
Extreme Programming	197
Scrum	198
Kanban	200
Lean Development.....	200
Requirements Engineering for Agile Methodologies	201
Technical Debt	205
Example Application of Agile Software Development	206
When Is Agile Recommended?	207
Requirements Engineering in XP	208
Requirements Engineering in Scrum.....	209
Gathering User Stories	210
Writing User Stories	210
Estimating User Stories	211
Prioritizing User Stories.....	212
User Stories vs. Use Cases.....	212
Agile Requirements Engineering vs. Requirements Engineering in Agile	212
Story-Test-Driven Development	212
State of Practice for Requirements Engineering in Agile Methodologies.....	214
Challenges for Requirements Engineering in Agile Methodologies	214
Exercises.....	216
Notes	217
References	217
9 Tool Support for Requirements Engineering	219
Introduction.....	219
Traceability Support.....	221
Requirements Linkage Traceability Matrix	223
Requirements Source Traceability Matrix	225
Requirements Stakeholder Traceability Matrix	226
Requirements Management Tools	227
Tool Evaluation.....	228
Open-Source Requirements Engineering Tools.....	229
FreeMind.....	230

FitNesse	232
Requirements Engineering Tool Best Practices	233
Elicitation Support Technologies	234
Using Wikis for Requirements Elicitation	234
Mobile Technologies	235
Virtual Environments	235
Content Analysis	236
Artificial Intelligence	237
Requirements Metrics	237
Exercises	238
References	239
10 Requirements Management	243
Introduction	243
Requirements Management Process	243
Configuration Management and Control	245
Reconciling Differences	245
Managing Divergent Agendas	246
Consensus Building	247
Analytical Hierarchical Process (AHP)	247
Wideband Delphi Technique	249
Expectation Revisited: Pascal's Wager	250
Global Requirements Management	251
Antipatterns in Requirements Management	253
Environmental Antipatterns	254
Divergent Goals	254
Process Clash	255
Management Antipatterns	255
Metric Abuse	255
Mushroom Management	256
Other Paradigms for Requirements Management	257
Requirements Management and Improvisational Comedy	257
Requirements Management as Scriptwriting	258
Standards for Requirements Management	260
Capability Maturity Model Integration	261
ISO 9001	262
ISO/IEEE 12207	263
Six Sigma	263
Exercises	266
Notes	266
References	266
11 Value Engineering of Requirements	269
What, Why, When, and How of Value Engineering	269
What Is Value Engineering?	270
When Does Value Engineering Occur?	271
Challenges to Simple Cost vs. Risk Analysis	272

Estimating Software Effort	273
The Aspect of Software Size.....	273
Software Length.....	274
Software Complexity.....	274
Functional Size	275
Estimating Using Function Points.....	275
Function Point Cost Drivers	275
Estimating Using COSMIC.....	277
The Relationship between Functional Size and Effort.....	277
Estimating Using COCOMO and Its Derivatives	279
COCOMO	279
WEBMO.....	281
COSYSMO	282
Feature Points	283
Use Case Points.....	284
Considerations for Nonfunctional Requirements in Size Measurements	284
Software Effort Estimation in Practice.....	285
Requirements Feature Cost Justification	287
Return on Investment.....	287
Net Present Value.....	288
Internal Rate of Return	288
Profitability Index	289
Payback Period.....	289
Discounted Payback.....	290
Putting It All Together	290
Exercises.....	292
Note.....	293
References	293
12 Requirements Engineering: A Road Map to the Future	295
Shaping Factors of the Future Landscape of Requirements Engineering	295
The Changing Landscape of Requirements Engineering.....	296
Requirements Engineering for Small Businesses	296
Requirements Engineering and Disruptive Technologies	298
Requirements Engineering and Blockchain.....	298
Requirements Engineering and Internet of Things	300
Requirements Engineering and Artificial Intelligence (AI).....	302
Requirements Engineering and Cloud Computing	303
Requirements Engineering and Affective Computing.....	304
References	304
Appendix A: Software Requirements Specification for a Smart Home	307
Appendix B: Software Requirements for a Wastewater Pumping Station	
Wet-Well Control System	329
Appendix C: Unified Modeling Language (UML)	341
Appendix D: User Stories	355

Appendix E: Use Cases.....363

Appendix F: IBM DOORS Requirements Management Tool.....381

Glossary.....389

Index395



Taylor & Francis

Taylor & Francis Group

<http://taylorandfrancis.com>

Preface

Solid requirements engineering has increasingly been recognized as the key to improved, on-time, and on-budget delivery of software and systems projects. Nevertheless, a few undergraduate engineering programs stress the importance of this discipline. Recently, however, some software and systems engineering programs have introduced requirements engineering as mandatory in their curricula. In addition, new software tools are emerging that are empowering practicing engineers to improve their requirements engineering habits. However, these tools are not usually easy to use without significant training, and many working engineers are returning for additional courses that will help them understand the requirements engineering process.

This book is intended to provide a comprehensive treatment of the theoretical and practical aspects of discovering, analyzing, modeling, validating, testing, and writing requirements for systems of all kinds, with an intentional focus on software-intensive systems. This book brings into play a variety of formal methods, social models, and modern requirements writing techniques to be useful to the practicing engineer.

Audience

This book is intended for professional software engineers, systems engineers, and senior and graduate students of software or systems engineering. Much of the material is derived from the graduate-level “Requirements Engineering” course taught at Penn State’s Great Valley School of Graduate and Professional Studies and online through its World Campus, where the authors work. The typical student in that course has 5 years of work experience as a software or systems engineering professional and an undergraduate degree in engineering, science, or business. Typical readers of this book will have one of the following or similar job titles:

Software engineer

Systems engineer

Sales engineer

Systems analyst

[XYZ] engineer (where “XYZ” is an adjective for most engineering disciplines, such as “electrical” or “mechanical”)

Project manager

Business analyst

Technical architect

Lead architect

Product Owner

Many others can benefit from this text including the users of complex systems and other stakeholders.

Exemplar Systems

Although various exemplar systems will be used to illustrate points throughout the book, four particular systems will be used repeatedly. These systems were selected because they involve application domains with which most readers are likely to be familiar and because they cover a wide range of applications from embedded to organic in both industrial and consumer implementations. Consider a domain, however, with which you (and the authors) are likely unfamiliar, say, mining of anthracite coal. Imagining the obvious difficulties in trying to communicate about such a system highlights the importance of domain understanding in requirements engineering. This topic will be discussed at length in Chapter 1.

The first of these running examples is an airline baggage handling system, probably similar to one found in every major airport. Check-in clerks and baggage handlers tag your bags at check-in with a barcode ID tag. Then the baggage is placed on a conveyor belt where it moves to a central exchange point and is redirected to the appropriate auxiliary conveyor for loading on an airplane-bound cart or a baggage carousel. Along the way, the system may conduct some kind of image scan and processing to detect the presence of unauthorized or dangerous contents (such as weapons or explosives). A baggage handling system is an embedded, real-time system, that is, the software is closely tied to the hardware, and deadline satisfaction is a key goal of the system.

The Denver International Airport tried to build a very sophisticated version of such a system several years ago. The system used PCs, thousands of remote-controlled carts, and a 21-mile-long track. Carts moved along the track, carrying luggage from check-in counters to sorting areas and then straight to the flights waiting at airport gates. After spending \$230 million over 10 years, the project was canceled (de Neufville 1994). Much of the failure can be attributed to requirements engineering mistakes.

The second exemplar system is a point of sale (POS) system for a large pet store chain. This type of system would provide such capabilities as cashier functions and inventory tracking, tax reporting, and end-of-year closeout. It might handle self-checkout, coupon scanning, product returns, and more. This business domain is transaction-oriented and although there are many available off-the-shelf systems let's assume there is a need for a custom system. This type of system is organic, that is, it is not tied to any specialized hardware. PCs or PC-based cash registers, storage devices, and network support comprise the main hardware.

The third system that will be explored is for a "Smart Home," that is, a home in which one or more PCs control various aspects of the home's climate control, security, ambiance, entertainment, and so forth. A Smart Home is a consumer application and a semi-detached system—it uses some specialized but off-the-shelf hardware and software. We will imagine that this Smart Home is being built for someone else (not you) so that you can remain objective about its features.

The fourth exemplar is a wet-well pumping system for a wastewater treatment station. Well-pumping systems, which are found in industrial and residential applications, are commonly used to illustrate various concepts in embedded, real-time systems control. The third and fourth systems are described in Appendices A and B, which include comprehensive (though imperfect) requirements specifications.

For the purposes of experimentation and practices, or for use in a course project, you are encouraged to select another appropriate system. Possible choices include:

Passenger safety restraint system for an automobile of your choice.

Family tree maker and genealogical database system.

Game of your choosing (keep it simple—3-D chess or checkers or popular board games are best).

Simulation of some familiar enterprise (e.g., a traffic intersection, elevator control, manufacturing environment).

High-level requirements for a safety-critical system such as a power plant.

Any aspect of your work that seems suitable to automation.

You can use your imagination, consult the many resources that are available on the Web, and have fun as you learn to “scope out” one or another of these systems.

Course Adoption

This book was written to support both undergraduate and graduate requirements engineering courses. The authors and their colleagues have used previous editions of this book in more than 50-course offerings at Penn State. Previous editions of the book have also been adopted by instructors at numerous universities worldwide.

Each chapter includes simple, intermediate, and advanced exercises. Advanced exercises are suitable as a research assignment or Independent study and are denoted by an asterisk (*). Vignettes at different locations of each chapter provide mini-case studies showing how the learnings in the chapter can be employed in real systems.

Sample requirements specifications are found in Appendices A and B. The first is for a smart home system. The second specification is for a well-pumping system. Those working in real-time systems or control engineering will be familiar with the widespread use of a well-pumping system in papers and texts. Neither of these requirements specifications is perfect nor complete. But they are used throughout the text for discussion, examples, and exercises. A possible course assignment could involve improving one of the specifications based on lessons from the text.

For those interested in adopting this text for an undergraduate or graduate course in requirements engineering, the following describes how the authors use the text in their courses (Kilcay-Ergin and Laplante 2012). For instructors adopting this text for courses in either a university or industrial setting, the authors can provide lecture notes upon request. Please contact them at (Philip Laplante, plaplante@psu.edu) or (Mohamad Kassab, muk36@psu.edu) for further information.

New for the Fourth Edition

Since the first edition, there have been made many changes and improvements. Feedback from instructors, students, and corporate users of the text was used to correct, expand, and improve the materials. Notable additions in the fourth edition include two newly added chapters: “On Non-Functional Requirements” (Chapter 5) and “Requirements Engineering: Road Map to the Future” (Chapter 12). The latter provides a discussion on the relationship between requirements

engineering and emerging disruptive technologies (Internet of Things, Cloud Computing, Blockchain, AI, and Affective Computing).

All chapters of the book were significantly expanded with new materials that keep the book relevant to the current industrial practices. Particularly, the reader will find expanded discussions on new elicitation techniques, agile approaches (e.g., Kanban, SAFe, DEVOps), requirements tools, requirements representation, risk management approaches, and functional size measurement methods. The provided statistics on RE state of practices were also updated throughout the book with results derived from a 2020 survey conducted by the authors. Besides, the fourth edition also has significant additions of vignettes, exercises, references throughout the book. Of course, this edition also implemented corrections and the removal of outdated material from the previous edition.

Throughout the book, the reader will also find newly added scannable QR codes that will direct the reader to one of the following:

- **Stay Updated:** a link to where the reader may find more up-to-date content when they become available.
- **Share Your Opinion:** the reader is invited to share an opinion (or experience) on selected topics.
- **Quick Access:** a link to a quick summary on a topic or quick access to a tool.
- **Supplemental Materials:** a link to supplemental materials by the authors on the selected topic.
- **Watch:** a link to a short video on a selected topic.

Requirements engineering is a dynamic field and the authors intend that this text keep pace with these changes. If you have any ideas for new material to be included in future additions of the book, please send these to the authors at (Phillip Laplante, plaplante@psu.edu) or (Mohamad Kassab, muk36@psu.edu).

Notes on Referencing and Errors

The authors have tried to uphold the highest standards for giving credit where credit is due. Each chapter contains a list of related readings, which have been extensively updated for the fourth edition, and they should be considered the primary references for that chapter. Where direct quotes or nonobvious facts are used, an appropriate note or in-line citation is provided. In particular, it is noted where the authors published portions in preliminary form in other scholarly publications.

Despite these best efforts and those of the reviewers and publisher, there are still likely errors to be found. Therefore, if you believe that you have found an error—whether it is a referencing issue, factual error, or typographical error, please contact the authors at the emails addressed referenced previously. You can access the list of known errata at <https://phil.laplante.io/requirements/errata.php>.



Disclaimers

When discussing certain proprietary systems, every effort has been taken to disguise the identities of any organizations as well as individuals that are involved. In these cases, the names and even elements of the situation have been changed to protect the innocent (and guilty). Therefore, any similarity between individuals or companies mentioned herein is purely coincidental.

REFERENCES

- de Neufville, R. (1994, December) The baggage system at Denver: Prospects and lessons. *Journal of Air Transport Management*, 1(4): 229–236.
- Kilicay-Ergin, N., & Laplante, P. A. (2012) An online graduate requirements engineering course. *IEEE Transactions on Education*. Available <http://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=6253277&isnumber=4358717>



Taylor & Francis

Taylor & Francis Group

<http://taylorandfrancis.com>

Acknowledgments

Many individuals assisted during the preparation of the first, second, and third editions of this text and the minor revised versions in between. Several people are especially noteworthy.

Dr. George Hacken of the New York Metropolitan Transportation Authority contributed many ideas on formal methods used in Chapter 7. The late Professor Larry Bernstein of Stevens Institute of Technology reviewed the first edition and provided encouragement and many suggestions for improvement. Dr. Colin Neill and Dr. Raghu Sangwan of Penn State read portions of the first edition and provided valuable feedback in many discussions subsequently. Dr. Nil Kilcay-Ergin provided meaningful feedback that influenced parts of the second edition.

Several former students of the authors made significant contributions. For example, Brad Bonkowski contributed to the case study found in Appendix A and Christopher Garrell contributed to the case study in Appendix B. Nicholas Natale contributed to the vignette at the end of Chapter 3. Many other students have read drafts of portions of the text and provided ideas, exercises, critical feedback, and examples. They are too numerous to name individually but the authors wish to thank them all.

Many of the appendices were contributed by professional colleagues of the authors, either directly to this edition, or through the *Encyclopedia of Software Engineering* (Taylor & Francis Publishing, 2010), of which Dr. Laplante was editor-in-chief. In particular, Appendix C on the Universal Modeling Language (UML) was contributed through the Encyclopedia by Jim Rumbaugh. Similarly, Appendix D on User Stories was written for the Encyclopedia by Don Shafer. Appendix E on Use Cases by Emil Vassev also first appeared in the Encyclopedia. Finally, Gavin Arthurs of IBM provided the excellent overview of DOORS NG found in Appendix F.

Various users of previous versions of this text, including students, professors, and professionals, have pointed out errors and suggestions for improvement—these individuals are also too numerous to mention, but the authors are grateful to them for helping to improve this book.

Of course, any errors of commission or omission that remain are due to the authors alone. Please contact them at:

Philip Laplante: plaplante@psu.edu, or

Mohamad Kassab: muk36@psu.edu

The authors would also like to thank the staff at Taylor & Francis, especially our editor John Wyzalek.

On personal notes, Dr. Laplante expresses his deepest gratitude to Nancy, Chris, Charlotte, Henry, and Miles for providing love, companionship, and encouragement during the writing of this text, while Dr. Kassab acknowledges with gratitude the support and love provided by his family members—Hassan, Mariam, Essa, Eman, Sana, and Samar—during the journey of writing this fourth edition.

Phil Laplante and Mohamad Kassab, October 2021



Taylor & Francis

Taylor & Francis Group

<http://taylorandfrancis.com>

Authors

Phillip A. Laplante is Professor of Software and Systems Engineering and a member of the graduate faculty at The Pennsylvania State University. His research, teaching, and consulting focus on software quality particularly with respect to requirements, testing, and project management. Before joining Penn State, he was a professor and senior academic administrator at other colleges and universities.

From 2010 to 2016, he was the founding chair of the Software Professional Engineer Licensure Committee for the National Council of Examiners of Engineers and Surveyors. This volunteer committee created maintains and scores the exam used throughout the United States to license Professional Software Engineers.

Dr. Laplante has consulted to Fortune 500 companies, startup ventures, the U.S. Department of Defense, NASA, and the National Institute for Standards and Technology (NIST). He is on the Board of Directors for a \$100 million heavy infrastructure construction company and serves on various corporate technology advisory boards.

Prior to his academic career, Dr. Laplante spent nearly a decade as a software engineer and project manager working on avionics (including the Space Shuttle), CAD, and software test systems. He was also director of business development for a software consulting firm. He has authored or edited 37 books and more than 300 papers, articles, reviews, and editorials.

Dr. Laplante received his BS, MEng, and PhD in computer science, electrical engineering, and computer science, respectively, from Stevens Institute of Technology and an MBA from the University of Colorado at Colorado Springs. He is a licensed professional engineer in Pennsylvania and is a Certified Software Development Professional. He is a fellow of the IEEE and SPIE and a member of numerous professional societies and program committees. He has won international awards for his teaching, research, and service.



Watch: A short Introductory video from Dr. Laplante
<https://phil.laplante.io/requirements/phil.intro.php>

Mohamad H. Kassab is an associate research professor and a member of the graduate faculty at The Pennsylvania State University. He earned his PhD and MS degrees in computer science from Concordia University in Montreal, Canada. Dr. Kassab was an affiliate assistant professor in the Department of Computer Science and Software Engineering at Concordia University between 2010 and 2012, a postdoctoral researcher in software engineering at Ecole de Technologie Supérieure (ETS) in Montreal between 2011 and 2012, and a visiting scholar at Carnegie Mellon University (CMU) between 2014 and 2015.

Dr. Kassab has been conducting research projects jointly with the industry to develop formal and quantitative models to support the integration of quality requirements within software and systems development life cycles. The models are being further leveraged with the support of developed architectural frameworks and tools. His research interests also include bridging the gap between software engineering practices and disruptive technologies (e.g., IoT, blockchain). He has published extensively in software engineering books, journals, and conference proceedings. He is also a member of numerous professional societies and program committees, and the organizer of many software engineering workshops and conference sessions.

With over 20 years of global industry experience, Dr. Kassab has developed a broad spectrum of skills and responsibilities in many software engineering areas. Notable experiences include business unit manager at Soramitsu, senior quality engineer at SAP, senior quality engineer at McKesson, senior associate at Morgan Stanley, senior quality assurance specialist at NOKIA, and senior software developer at Positron Safety Systems. He is an Oracle Certified Application Developer, Sun Certified Java Programmer, and Microsoft Certified Professional.

Dr. Kassab has taught a variety of graduate and undergraduate software engineering and computer science courses at Penn State and Concordia University. He has won many awards for his excellence in teaching.

Watch: A short Introductory video from Dr. Kassab
<https://phil.laplante.io/requirements/mohamad.intro.php>



Chapter 1

Introduction to Requirements Engineering

Motivation

Very early in the drive to industrialize software development, Royce (1975) pointed out the following truths:

There are four kinds of problems that arise when one fails to do adequate requirements analysis: top-down design is impossible; testing is impossible; the user is frozen out; management is not in control. Although these problems are lumped under various headings to simplify the discussion, they are all variations of one theme—poor management. Good project management of software procurement is impossible without some form of explicit (validated) and governing requirements.

Even though the top-down design approach has been largely replaced by object-oriented design, these truths still apply today.

A great deal of research has verified that devoting systematic effort to requirements engineering can greatly reduce the amount of rework needed later in the life of the software or software-intensive product and can cost-effectively improve various qualities of the system. Too often systems engineers forego sufficient requirements engineering activity either because they do not understand how to do requirements engineering properly, or because there is a rush to start coding (in the case of a software product). Clearly, these eventualities are undesirable, and it is a goal of this book to help engineers understand the correct principles and practices of requirements engineering.

What Is Requirements Engineering?

There are many ways to portray the discipline of requirements engineering depending on the viewpoint of the definer. For example, a bridge is a complex system but has a relatively small number

of patterns of design that can be used (e.g., suspension, trussed, cable-stayed). Bridges also have specific conventions and applicable regulations in terms of load requirements, materials that are used, and the construction techniques employed. So, when speaking with a customer (e.g., the state department of transportation) about the requirements for a bridge, much of its functionality can be captured succinctly:

The bridge shall replace the existing span across the Brandywine River at Creek Road in Chadds Ford, Pennsylvania, and shall be a cantilever bridge of steel construction. It shall support two lanes of traffic in each direction and handle a minimum capacity of 100 vehicles per hour in each direction.

Of course, there is a lot of information missing from this “specification” (e.g., weight restrictions), but it substantially describes what this bridge is to do and the design choices available to achieve these requirements are relatively simple.

Other kinds of systems, such as biomechanical or nanotechnology systems with highly specialized domain language, have seemingly exotic requirements and constraints. Still, other complex systems have so many kinds of behaviors that need to be embodied (even word processing software can support thousands of functions) that the specification of said systems becomes very challenging indeed.

Since the authors are primarily software engineers, we refer to that discipline for a convenient, more-or-less universal definition for requirements engineering that is due to Pamela Zave:

Requirements engineering is the branch of software engineering concerned with the *real-world goals* for, functions of, and constraints on software systems. It is also concerned with the relationship of these factors to *precise specifications* of software behavior, and to their *evolution over time and across software families*. (Zave 1997)

But we wish to generalize the notion of requirements engineering to include any system, whether it be software only, hardware only, or hardware and software (and many complex systems are a combination of hardware and software), so we rewrite Zave’s definition as follows:

Requirements engineering is the branch of **engineering** concerned with the *real-world goals* for, functions of, and constraints on **systems**. It is also concerned with the relationship of these factors to *precise specifications* of **system** behavior and to their evolution over time and across families of **related systems**.

The changes we have made to Zave’s definition are in bold. We refer to this modified definition when we speak of “requirements engineering” throughout this text. Though some may argue that the term “Requirements Engineering” is a misnomer, the term “engineering” serves as a reminder that RE is an important part of software/system development that is concerned with anchoring development to solve a real-world problem. We will explore all the ramifications of this definition and the activities involved in great detail as we move forward.

You Probably Don’t Do Enough Requirements Engineering

Research suggests that requirements engineering is not done well by the industry. For example, in a global survey of more than 3,000 practicing engineers (over 250 respondents), 37% responded that

requirements engineering practices in their companies were less than satisfactory (Kassab et al. 2014). In a more recent survey (Kassab and Laplante 2022), the number has slightly decreased to 30%.



Stay Updated: For the up-to-date data from the RE state of practice survey.
<https://phil.laplante.io/requirements/updates/survey.php>

Another study of corporate IT projects performed by IAG Consulting found that of the companies surveyed, fully two-thirds of the respondents classified their projects as likely failures. The most frequently cited reason for failure by respondents was poor requirements gathering techniques (Kanaracus 2008). In a third study of seven embedded systems engineering companies in Europe, Sikora et al. (2012) found similar results; in particular, they concluded that “existing requirements engineering methods are insufficient for handling requirements for complex embedded systems.”

One potential reason for inadequate requirements engineering is suggested by Wnuk et al. (2011) who observed that companies seem to have difficulty in scaling up requirements engineering practices in terms of activity scope and the structure of requirements artifacts. In their survey, Sikora et al. (2012) found another possible reason for the apparent inadequacy in industrial requirements engineering. They found that practitioners desire “requirements specifications that are properly integrated into the overall systems architecture but are solution-free with regard to the specified function or component itself.” Survey respondents noted that existing method support is inadequate in supporting this goal. It seems, then, that requirements engineering has a long way to go in meeting the needs of practitioners.

The lack of proper requirements engineering practices is even more widespread in small businesses (Kassab 2021)—even though proper requirements engineering practices can mean the difference between survival or failure. We will return back to the discussion on requirements engineering in small businesses in Chapter 12.

What Are Requirements?

Part of the challenge in requirements engineering has to do with an understanding of what a “requirement” really is. Requirements can range from high-level, abstract statements, and back-of-the-napkin sketches to formal (mathematically rigorous) specifications. These varying representation forms occur because stakeholders have needs at different levels and hence depend on different abstraction representations. Stakeholders also have varying abilities to make and read these representations (e.g., a business customer vs. a design engineer), leading to diverse quality in the requirements. We will discuss the nature of stakeholders and their needs and capabilities in the next chapter.

Requirements vs. Features vs. Goals

A fundamental challenge for the requirements engineer is recognizing that customers often confuse requirements, features, and goals (and engineers sometimes do too).

While *Goals* are high-level objectives of a business, organization, or system, a requirement specifies how a goal should be accomplished by a proposed system. On the other hand, a feature is a set of logically related requirements that allows the user to satisfy the goal.

So, to the users of a “Smart home system,” one goal is to automate that which does not need human interaction, to free the occupants to enjoy themselves with other activities. This goal can be satisfied by implementing some features. One of these features could be by providing automated water purification to the system. Then, there will be a set of specific requirements to describe the water purification mechanism (see Appendix A). So, the requirements tend to be more granular than a feature and tend to be written with the implementation in mind.

To treat a goal as a requirement is to invite trouble because the achievement of the goal will be difficult to prove. In addition, goals evolve as stakeholders change their minds and refine and operationalize goals into behavioral requirements.

Requirements Classifications

There are many schemes available to classify requirements. We will discuss two popular schemes: *Requirements Level Classification* and *Requirements Specifications Types*.

Requirements Level Classification

To deal with the diversity in requirements types, Sommerville (2005) suggests organizing them into three levels of abstraction:

- User requirements
- System requirements
- Design specifications

User requirements are abstract statements written in natural language with accompanying informal diagrams. They specify what services (user functionality) the system is expected to provide and any constraints. Collected user requirements often appear as a “concept of operations” (Conops) document. In many situations, user stories can play the role of user requirements.

System requirements are detailed descriptions of the services and constraints. System requirements are sometimes referred to as *functional specifications* or *technical annexes* (a term that is rarely used). These requirements are derived from analysis of the user requirements, and they should be structured and precise. The requirements are collected in a systems requirements specification (SRS) document. Use cases can play the role of system requirement in many situations.

Finally, design specifications emerge from the analysis and design documentation used as the basis for implementation by developers. The system design specification is essentially derived directly from the analysis of the system requirements specification.

To illustrate the differences in these specification levels, consider the following from the airline baggage handling system:

A user requirement

- The system shall be able to process 20 bags per minute.

Some related system requirements

- Each bag processed shall trigger a baggage event.
- The system shall be able to handle 20 baggage events per minute.

Finally, the associated system specifications

- 1.2 The system shall be able to process 20 baggage events per minute in operational mode.
 - 1.2.1 If more than 20 baggage events occur in a 1-minute interval, then the system shall ...
 - 1.2.2 [more exception handling] ...

For a pet store POS system, consider the following:

A user requirement

- The system shall accurately compute sale totals including discounts, taxes, refunds, and rebates; print an accurate receipt; and update inventory counts accordingly.

Some related system requirements

- Each sale shall be assigned a sales ID.
- Each sale may have one or more sales items.
- Each sale may have one or more rebates.
- Each sale may have only one receipt printed.

Finally, the associated software specifications

- 1.2 The system shall assign a unique sales ID number to each sale transaction.
 - 1.2.1 Each sales ID may have zero or more sales items associated with it, but each sales item must be assigned to exactly one sales ID

The systems specification in the appendices also contains numerous specifications organized by level for your exploration.

The different specification levels guide progressive testing throughout the project lifecycle (Figure 1.1).

Thus, user requirements, which are discovered first, are used as the basis for final acceptance testing. The systems requirements, which are generally developed after the user requirements, are used as the basis for the integration testing that precedes acceptance testing. And, finally, the design specifications that are derived from the system requirements are used for unit testing as each code unit is implemented.

Requirements Specifications Types

Another taxonomy for requirements specifications focuses on the type of requirement from the following list of possibilities:

- Functional requirements (FRs)
- Nonfunctional requirements (NFRs)
- Domain requirements

Let's look at these more closely.

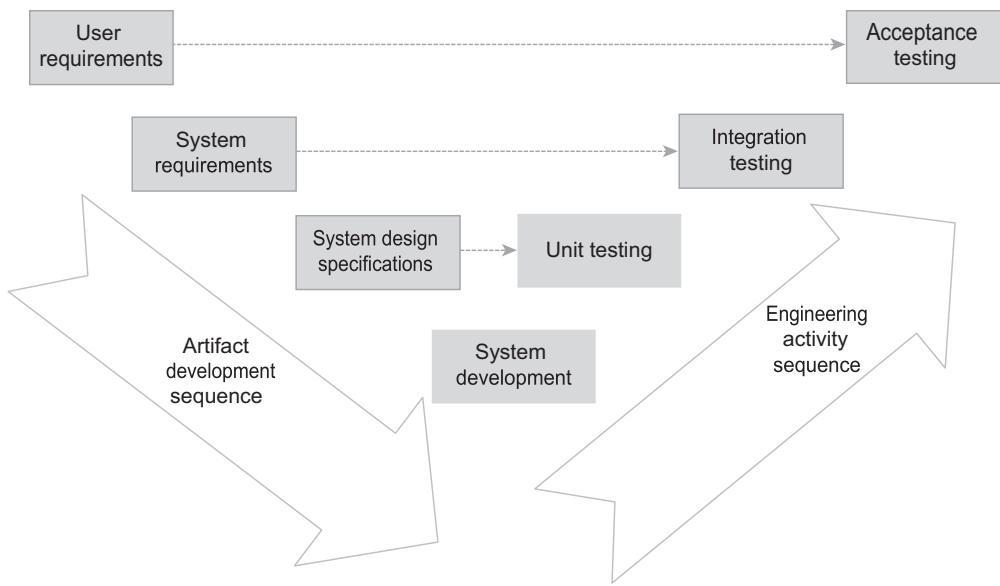


Figure 1.1 Relationship between requirements specification levels and testing.

Functional Requirements

Functional requirements (FRs) describe the services the system should provide and how the system will react to its inputs. In addition, FRs need to explicitly state certain behaviors that the system should not do (more on this later). FRs can be high level and general (in which case they are user requirements in the sense that was explained previously) or they can be detailed, expressing inputs, outputs, exceptions, and so on (in which case they are the system requirements described before).

There are many forms of representation for FRs, from natural language (in our case, the English language), visual models, and the more rigorous formal methods. We will spend much more time discussing requirements representation in Chapter 4.

To illustrate some FRs, consider the following sampling of ones for the baggage handling system.

- 2.1 The system shall handle up to 20 bags per minute. ...
- 2.4 When the system is in idle mode, the conveyor belt shall not move. ...
- 2.8 If the main power fails, the system shall shut down in an orderly fashion within 5 seconds. ...
- 2.41 If the conveyor belt motor fails, the system shall shut down the input feed mechanism within 3 seconds.

For the pet store POS system, the following might be some FRs:

- 4.1 When the operator presses the “total” button, the current sale enters the closed-out state.
 - 4.1.1 When a sale enters the closed-out state, a total for each non-sale item is computed as the number of items times the list price of the item.
 - 4.1.2 When a sale enters the closed-out state, a total for each sale item is computed.

More FRs can be found in Section 3 of Appendix A and Section 2 of Appendix B.

Nonfunctional Requirements

While systems are characterized by functional behavior (what the system does). They are also characterized by their nonfunctional behavior (how the system behaves concerning some observable attributes like reliability, reusability, maintainability). In the software/system marketplace, in which functionally equivalent products compete for the same customer, nonfunctional requirements (NFRs) become more important in distinguishing between the competing products. They basically deal with the following issues:

- Security
- Reliability
- Maintainability
- Performance
- Usability
- Testability
- Interoperability
- Constraints (e.g., design/implementation constraints, financial constraints, resources constraints)

We will discuss NFRs in more detail in Chapter 5.

Domain Requirements

Domain requirements are derived from the application domain. These types of requirements may consist of new FRs or constraints on existing FRs, or they may specify how particular computations must be performed.

In the baggage handling system, for example, various domain realities create requirements. There are industry standards (we wouldn't want the new system to underperform vs. other airlines' systems). There are constraints imposed by existing hardware available (e.g., conveyor systems). And there may be constraints on performance mandated by collective bargaining agreements with the baggage handlers union.

For the pet store POS system, domain requirements are imposed by conventional store practices. For example:

- Handling of cash, credit cards, and coupons
- Display interface and receipt format
- Conventions in the pet store industry (e.g., frequent-buyer incentives, buy one get one free)
- Sale of items by the pound (e.g., horse feed) vs. by item count (e.g., dog leashes)

Appendices A and B also contain other examples of NFRs.

Domain Vocabulary Understanding

The requirements engineer must be sure to fully understand the application domain vocabulary (or have someone fluent in that vocabulary at the ready) as there can be subtle and profound differences in the use of terms in different domains. The following true incident illustrates the point. The first author was once asked to provide consulting services to a very

large, international package delivery company. After a few hours of communicating with the engineers of the package delivery company, it became clear that the author was using the term “truck” incorrectly. While the author believed that the “truck” referred to the familiar vehicle that would deliver packages directly to customers’ homes, the company used the term “package car” to refer to those vehicles. The term “truck” was reserved to mean any long-haul vehicle (usually an 18-wheeled truck) that carried large amounts of packages from one distribution hub to another. So there was a huge difference in the volume of packages carried in a “truck” and a “package car.” Imagine, then, if a requirement was written involving the processing of “packages from 1,000 trucks” (when it was really meant “1,000 package cars”). Clearly, this difference in domain terminology understanding would have been significant and potentially costly.

VIGNETTE 1.1 Requirements for a Brick

The requirements engineer is sometimes challenged with the need for, or value of, requirements engineering activities, even the purpose of a requirements specification. The following is a real incident. A consulting requirements engineer was asked to provide guidance to the developers of a very large, complex, multimillion-dollar automated materials handling system. The system was already partially built but there were serious breakdowns in communications between the hardware and software engineers, jeopardizing the project. The consulting requirements engineer asked to see the requirements specification—but there was none. The chief engineer did not see the need for a specification document beyond a set of evolving Kanban cards (see chapter 8). To demonstrate the irrelevance of a requirements specification he challenged: “what are the requirements for a brick,” hoping to trap the consulting requirements engineer into admitting that no such specifications exist. But there are standards-based requirements specifications for bricks. The ASTM International (formerly American Society for Testing and Materials) publishes technical standards for a wide range of materials, products, systems, and services, including bricks. ASTM C62-17 Standard Specification for Building Brick (Solid Masonry Units Made From Clay or Shale) specifies size, appearance, durability, strength, and more for bricks of all kinds (<https://www.astm.org/Standards/C62.htm>). This standard and the requirements derived from it are very important. For example, if nonstandard bricks lacked the expected compressive strength or weathering properties, a brick home might not be able to withstand severe weather conditions. Even unwanted and unexpected variations in size and coloration could be a problem for customers. So, yes, requirements are needed for even a system as simple as a brick, and well-developed requirements specifications are certainly needed in complex, multimillion-dollar systems. The chief engineer was embarrassed and unhappy with the consulting requirements engineer’s answer because it exposed a serious project weakness. The consulting requirements engineer was never called back, and at this writing, the complex material handling system has not been deployed.

Requirements Engineering Activities

The requirements engineer is responsible for a number of activities. These include:

- Requirements elicitation/discovery
- Requirements analysis and reconciliation
- Requirements representation/modeling
- Requirements verification and validation
- Requirements management

We explore each of these activities briefly in the following sections and substantially more so in subsequent chapters.

Requirements Elicitation/Discovery

Requirements elicitation/discovery involves uncovering what the customer needs and wants. But elicitation is not like harvesting low-hanging fruit from a tree. While some requirements will be obvious (e.g., the POS system will have to compute sales tax), many requirements will need to be extricated from the customer through well-defined approaches. This aspect of requirements engineering also involves discovering who the stakeholders are; for example, are there any hidden stakeholders? Elicitation also involves determining the NFRs, which are often overlooked.

Requirements Analysis and Agreement

Requirements analysis and requirements agreement involve techniques to deal with a number of problems with requirements in their “raw” form, that is, after they have been collected from the customers. Problems with raw requirements include the following:

- They don’t always make sense.
- They often contradict one another (and not always obviously so).
- They may be inconsistent.
- They may be incomplete.
- They may be vague or just wrong.
- They may interact and be dependent on each other.

Many of the elicitation techniques that we will discuss subsequently are intended to avoid or alleviate these problems. Formal methods are also useful in this regard. Requirements analysis and agreement are discussed in Chapter 4.

Requirements Representation

Requirements representation (or modeling) involves converting the requirements processed raw requirements into some model (usually natural language, mathematics, and visualizations). Proper representations facilitate communication of requirements and conversion into a system architecture and design. Various techniques are used for requirements representation including informal (e.g., natural language, sketches, and diagrams), formal (mathematically sound representations),

and semiformal (convertible to a sound representation or can be made fully formal by the addition of a semantic framework). Usually, some combination of these is employed in requirements representation, and we will discuss these further in Chapters 4 and 7.

Requirements Validation

Requirements validation is the process of determining if the specification is a correct representation of the customers' needs. Validation answers the question "Am I building the right product?" Requirements validation involves various semiformal and formal methods, text-based tools, visualizations, inspections, and so on, and is discussed in Chapter 6.

Requirements Management

One of the most overlooked aspects of requirements engineering, requirements management involves managing the realities of changing requirements over time. It also involves fostering traceability through appropriate aggregation and subordination of requirements and communicating changes in requirements to those who need to know.

Managers also need to learn the skills to intelligently push back when scope creep ensues. Using tools to track changes and maintain traceability can significantly ease the burden of requirements management. We will discuss software tools to aid requirements engineering in Chapter 9 and requirements management overall in Chapter 10.

Bodies of Knowledge

Three important structured taxonomies or bodies of knowledge exist for requirements engineering of software systems: the Software Engineering Body of Knowledge Version 3.0 (SWEBOK 2014), the Graduate Software Engineering Reference Curriculum (GSwE 2009), and the Principles and Practices (P&P) of Software Engineering Exam Specification (Kilcay-Ergin and Laplante 2013). These bodies of knowledge focus on the discipline of software engineering, but they can be applied to the engineering of any kind of system, whether software, electrical, mechanical, or hybrid.

SWEBOK was established to promote a consistent view of software engineering and to provide a foundation for curriculum development and for certification and licensing examinations. SWEBOK identifies the fundamental skills and knowledge that all graduates of a master's program in software engineering must possess.

The GSwE report highlights the strong link between software engineering and systems engineering and suggests that system engineering knowledge areas should be integrated into the software engineering curriculum (Kilcay-Ergin and Laplante 2013).

The Software Engineering P&P exam is used by states and jurisdictions of the United States as one component of licensure for those individuals that are working on software systems that affect the health, safety, and welfare of the public (Laplante et al. 2013).

Since the intent of the taxonomies is different, the scope of requisite knowledge also differs in some ways; for example, there is no clear coverage of GSwE's initiation and scope definition in SWEBOK. A comparison of the scope of knowledge for these bodies of knowledge is shown in Table 1.1.

This text is intended to provide reasonable coverage of most of the topics from the SWEBOK; however, it substantially covers the other two bodies of knowledge.

Table 1.1 Comparison of GSwE, SWEBOK, and Software Engineering P&P Knowledge Areas

<i>GSwE 2009</i>	<i>SWEBOK Version 3.0</i>	<i>Software Engineering P&P</i>
Fundamentals of RE • Relationship between systems engineering and software engineering • Definition of requirements • System design constraints • System design and requirements allocation • Product and process requirements • Functional and nonfunctional requirements • Emergent properties • Quantifiable requirements	Software Requirements Fundamentals • Definition of a software requirement • Product and process requirements • Functional and nonfunctional requirements • Emergent properties • Quantifiable requirements • System requirements and software requirements	Software Requirements Fundamentals • Concept of operations • Types of requirements • Product and process requirements • Functional and nonfunctional requirements • Quantifiable requirements • System requirements • Software requirements • Derived requirements • Constraints, service level
RE Process • Process models • Process actors • Process support and management • Process quality and improvement	Requirements Process • Process models • Process actors • Process support and management • Process quality and Improvement	
Initiation and Scope Definition • Determination and negotiation of requirements • Feasibility analysis • Process for requirements review/revision	No equivalent	

(Continued)

Table 1.1 (Continued) Comparison of GSWE, SWEBOK, and Software Engineering P&P Knowledge Areas

<i>GSWE 2009</i>	<i>SWEBOK Version 3.0</i>	<i>Software Engineering P&P</i>
Requirements Elicitation - Requirements sources - Elicitation techniques	Requirements Elicitation - Requirements sources - Elicitation techniques	Requirements Elicitation - Requirements sources - Elicitation techniques - Requirements representation
Requirements Analysis - Requirements classification - Conceptual modeling - Heuristic methods - Formal methods - Requirements negotiation	Requirements Analysis - Requirements Classification - Conceptual modeling - Architectural design and requirements allocation - Requirements negotiation - Formal analysis	
Requirements Specification Requirements specification techniques	Requirements Specification - The System definition document - System requirements specification - Software requirements specification	Requirements Specification - System definition document - System/subsystems specification - Software requirements specification - Interface requirements specification
Requirements Validation - Requirements reviews - Prototyping - Model validation - Acceptance tests	Requirements Validation - Requirements reviews - Prototyping - Model validation - Acceptance tests	Requirements Verification and Validation - Requirements reviews - Prototyping - Model validation - Simulation
Practical Considerations - Iterative nature of requirements process - Change management - Requirements attributes - Requirements tracing - Measuring requirements	Practical Considerations - Iterative nature of the requirements process - Change management - Requirements attributes - Requirements tracing - Measuring Requirements	Requirements Management - Iterative nature of the requirements process - Change management - Requirement attributes - Requirements traceability - Measuring requirements - Software requirements tools

Source: Adapted from Kilcay-Ergin, N., & Laplante, P.A., *IEEE Trans. Educ.*, 56: 199–207, 2013.

Also worth mentioning are other proposals, reference curricula, and bodies of knowledge that also focus significantly on requirements engineering processes and activities. These include:

- The *Requirements Engineering Body of Knowledge (REBoK 2013)*,
- The *Project Management Body of Knowledge (PMBOK)*,

- The *Business Analysis Body of Knowledge (PABOK)*,
- The *Guide to the Systems Engineering Body of Knowledge (SEBoK 2012)*,
- The *Graduate Reference Curriculum for Systems Engineering (GRCSE 2012)*,
- The Association for Computing Machinery (ACM)/Institute for Electrical and Electronics Engineers (IEEE) Computer Society's *Curriculum Guidelines for Undergraduate Degree Programs in Software Engineering (ACM/IEEE 2014)*. These also focus significantly on requirements engineering processes and activities.

The Requirements Engineer

What skills should requirements engineer have?

Since the 1990s, labor market observers (e.g., CNN Money) have consistently reported the Business Analysts' job as one of the highest paid—not only in the IT sector but also in general. Recently, the US Bureau of Labor Statistics announced that the employment of analysts is “projected to grow 21% until 2024, much faster than the average for all occupations.” The growth in cloud computing, Artificial Intelligence (AI), the Internet of Things (IoT), cybersecurity, and mobile networks will only increase the demand for these professionals and is expected to outstrip the supply of talent. For example, the 2015–2019 report of Canada's ICT Council, a government-funded labor market intelligence and industry skills standard body, warns that “As a result of employment growth—combined with replacement demand due to skills mismatch, retirements, and other exits, demand-supply imbalances will affect some occupations more than others” and names “information systems analysts” as the first among the Top-7 so-called “high-demand occupations.” Despite these market developments, relatively little has been published on the “requirements engineers” occupation in terms of in-demand competencies, skills, and professional experience as perceived by companies employing RE specialists.

There are studies that investigated the occupational aspects of the requirements engineer role in Canada (Wang et al. 2018), Germany (Herrmann 2013), in the Netherlands (Daneva et al. 2017), and in Mexico and Brazil (Calazans et al. 2017).

While well-established practitioners' communities framed RE as a profession from the body-of-knowledge perspective (e.g., PMBOK, SWEBOK, GSwE, REBOK, and BABOK). Each of these reflects the collective knowledge of the respective community, presents their most widely accepted practices, and positions RE in a unique way, as per the interfaces of RE with the critical activities of the respective professional community. Al-Ani and Sim (2006) proposed a model of the key RE areas of expertise and matched them against RE activities.

Christensen and Chang suggest that the requirements engineer should be organized, have experience throughout the (software) engineering lifecycle, have the maturity to know when to be general and when to be specific, and be able to stand up to the customer when necessary (Christensen and Chang 1996). Christensen and Chang further suggest that the requirements engineer should be a good manager (to manage the process), a good listener, fair, a good negotiator, and multidisciplinary (e.g., have a background in traditional hard sciences and engineering augmented with communications and management skills). Finally, the requirements engineer should understand the problem domain.

Gorla and Lam (2004) argued that engineers should be thinking, sensing, and judging in the Myers-Briggs sense. We could interpret this observation to mean that requirements engineers are structured and logical (thinking), focus on information gathered and do not try to interpret it (sensing), and seek closure rather than leaving things open (judging).

Klendauer et al. (2012) derived a RE competence model integrating contextual and situational factors tied to RE results variables. Penzenstadler et al. (2013) categorized the soft skills required for the execution of specific RE activities.

Finally, Ebert (2010) suggests that requirement engineers should have competency in the following areas:

- Requirements engineering
- Systems engineering
- Management
- Communication
- Cognition
- Social interaction

Ebert further notes that academic programs are insufficient to develop these competencies and that these must be acquired through years of practice and study. Academic courses can, however, start new requirements engineers off in the right direction.

Requirements Engineer Roles

The collective conclusion from both scholars' and practitioners' sources is that there is a significant incongruity regarding the perceptions of the RE role, tasks, and responsibilities in the marketplace. A major source of confusion is that a requirements engineer can be acting in different roles. We have identified the following role models:

- Requirements engineer as a software systems engineer
- Requirements engineer as a subject matter expert (SME)
- Requirements engineer as an architect
- Requirements engineer as a business process expert
- The ignorant requirements engineer

There are hybrid roles for the requirements engineer from the above as well.

Requirements Engineer as Software or Systems Engineer

It is likely that many requirements engineers are probably software engineers, electrical engineers, or systems engineers. When this is the case, the requirements engineer can positively influence the downstream development of models (e.g., the software design). The danger, in this case, is that the requirements engineer may begin to create a design when he should be developing requirements specifications.

Requirements Engineer as Subject Matter Expert

In many cases, the customer is looking to the requirements engineer to be an SME for expertise either in helping to understand the problem domain or in understanding the customers' own wants and desires. Sometimes the requirements engineer isn't an SME—they are an expert in requirements engineering. In those cases where the requirements engineer is not an SME, consider joining forces with an SME.

Requirements Engineer as Architect

Building construction is often used as a metaphor for software construction. In the authors' experience, architects and landscape architects play similar roles to requirements engineers (and this similarity is often used as an argument that software engineers need to be licensed). Daniel Berry has written about this topic extensively (Berry 1999, 2003). Berry noted that the analogous activities reduce scope creep and better involve customers in the requirements engineering process. In addition, Zachman (1987) introduced an architectural metaphor for information systems, though his model is substantially different from the one about to be presented.

The similarities between architecture (in the form of home specification) and software/systems specification are summarized in Table 1.2. If you have been through the process of constructing or renovating a home, you will appreciate the similarities.

Requirements Engineer as Business Process Expert

The activities of requirements engineering comprise a form of problem solving—the customer has a problem and the system must solve it. Often, solving the problem at hand involves the requirements engineer advising changes in business processes to simplify the expression of system behavior. While it is not the requirements engineer's role to conduct business process improvement, this side benefit is frequently realized.

Ignorance as Virtue

Berry (1995) suggested having both novices and experts in the problem domain involved in the requirements engineering process. His rationale is as follows. The “ignorant” people ask the “dumb” questions, and the experts answer these questions. Having the requirements engineer as

Table 1.2 Architectural Model for Systems Engineering

<i>Homebuilding</i>	<i>Software/System Building</i>
Architect meets with and interviews clients. Tours property. Takes notes and pictures.	Requirements engineer meets with customers and uses interviews and other elicitation techniques.
Architect makes rough sketches (shows to clients, receives feedback).	Requirements engineer makes models of requirements to show to customers (e.g., prototypes, draft SRS).
Architect makes more sketches (e.g., elevations) and perhaps more sophisticated models (e.g., cardboard, 3D-virtual models, fly-through animations).	Requirements engineer refines requirements and adds formal and semiformal elements (e.g., UML). More prototyping is used.
Architect prepares models with additional detail (floor plans).	Requirements engineer uses information determined above to develop complete SRS.
Future models (e.g., construction drawings) are for contractors' use.	Future models (e.g., software design documents) are for developers' use.

the most ignorant person in the group is not necessarily a bad thing, at least with respect to the problem domain because it forces him to ask the hard questions and to challenge conventional beliefs. Of course, the ignorant requirements engineer is completely in opposition to the role of SME.

Berry also noted that using formal methods in requirements engineering is a form of ignorance because a mathematician is generally ignorant about an application domain before he or she starts modeling it.

Role of the Customer

What role should the requirements engineer expect the customer to play? The customers' roles are varied and include:

- Helping the requirements engineer understand what they need and want (elicitation and validation)
- Helping the requirements engineer understand what they don't want (elicitation and validation)
- Providing domain knowledge when necessary and possible
- Alerting the requirements engineer quickly and clearly when they discover that they or others have made mistakes
- Alerting the requirements engineer quickly when they determine that changes are necessary (really necessary)
- Controlling their urges to have "aha moments" that cause major scope creep
- Sticking to all agreements

In particular, the customer is responsible for answering the following four questions, with the requirements engineer's help, of course:

1. Is the system that I want feasible?
2. If so, how much will it cost?
3. How long will it take to build?
4. What is the plan for building and delivering the system?

The requirements engineer must manage customers' expectations with respect to these issues. We will explore the nature and role of customers and stakeholders in the next chapter.

Problems with Traditional Requirements Engineering

Traditional requirements engineering approaches suffer from a number of problems, many of which we have already addressed (and others, which will be addressed) and many of these are not easily resolved. These problems include:

- Natural language problems (e.g., ambiguity, imprecision)
- Domain understanding

- Dealing with complexity (especially temporal behavior)
- Difficulties in enveloping system behavior
- Incompleteness (missing functionality)
- Over-completeness (gold-plating)
- Overextension (dangerous “all”)
- Inconsistency
- Incorrectness
- and more

We will explore the resolution of these problems throughout the book and particularly in Chapter 6.

Natural language problems result from the ambiguities and context sensitivity of natural (human) languages. We know these language problems exist for everyone, not just requirements engineers, and lawyers and legislators make their living finding, exploiting, or closing the loop-holes found in any laws and contracts written in natural language.

We have already covered the issue of domain understanding and have observed, as have others, that the requirements engineer may be an expert in the domain in which the system to be built will operate. System complexity is a pervasive problem that faces all systems engineers, and this will be discussed shortly. The problems of fully specifying system behavior and of missing behavior are also very challenging, but some techniques can be helpful in missing, at least, the most obvious functionality in a system.

Complexity

One of the greatest difficulties in dealing with the requirements engineering activities, particularly elicitation and representation, for most systems is that they are complex. Without trying to invent a definition for complexity, we contend that the challenges and complexity of capturing nontrivial behavior of any kind illustrate the notion of complex. Such difficulties are found in even the simplest of repeatable human endeavors.

Imagine, for example, someone asked you to describe the first 5 minutes of your morning from the moment you wake up. Could you do it with precision (try it)? No, you could not. Why? There are too many possible different paths your activities can take and too much uncertainty. Even with an atomic clock, you can't claim to wake up at the same time every day (because even atomic clocks are imperfect). But of course, you wake up differently depending on the day of the week, whether you are on vacation from work, if it is a holiday, and so on. You have to account for that in your description. But what if you wake up sick? How does that change the sequence of events? What if you accidentally knock over the glass of water on your nightstand as you get up? Does that change the specification of the activity? Or if you trip on your dog as you head to the bathroom? We could go on with this example and repeat this exercise with other simple tasks such as mowing the lawn or shopping for food. In fact, until you constrain the problem to the point of ridiculousness, you will find it challenging or even impossible to precisely capture any nontrivial human activities.

Now consider complex information or embedded processing system. Such a system will likely have to have interactions with humans. Even in those systems that do not directly depend on human interaction, it is the intricacy of temporal behavior as well as the problems of unanticipated events that make requirements elicitation and specification so difficult (and all the other requirements activities hard too).

Rittel and Webber (1973) defined a class of complex problems that they called “wicked.” Wicked problems are complex problems that have ten characteristics:

- There is no definitive formulation of a wicked problem.
- Wicked problems have no stopping rule.
- Solutions to wicked problems are not true or false, but good or bad.
- There is no immediate and no ultimate test of a solution to a wicked problem.
- Every solution to a wicked problem is a “one-shot operation”; because there is no opportunity to learn by trial and error, every attempt counts significantly.
- Wicked problems do not have an enumerable (or an exhaustively describable) set of potential solutions, nor is there a well-described set of permissible operations that may be incorporated into the plan.
- Every wicked problem is essentially unique.
- Every wicked problem can be considered a symptom of another problem.
- The existence of a discrepancy representing a wicked problem can be explained in numerous ways. The choice of explanation determines the nature of the problem’s resolution.
- The planner (designer) has no right to be wrong.

Rittel and Webber meant wicked problems to be of an economic, political, and societal nature (e.g., hunger, drug abuse, and conflict in the Middle East); therefore, they offer no appropriate solution strategy for requirements engineering. Nevertheless, it is helpful to view requirements engineering in the context of a wicked problem because it helps explain why the task is so difficult—because in many cases, real systems embody many of the characteristics of a wicked problem.

Even though “wicked” problems are complex problems, not every complex problem is a “wicked” problem. A requirements specification for a 3-D printing system which can print artificial bone components quickly during replacement surgery or an autonomous vehicle, are complex but these kinds of systems are already in development or deployed so their creation cannot be a wicked problem.

Gold-Plating and Ridiculous Requirements

In his outstanding book on systems design, Brooks (2010) cautions against “ridiculous requirements,” that is requirements that would be difficult to deliver given the state of technology at the time they are proposed. Another kind of ridiculous requirement would be one that, though possible to realize, is unlikely to be used.

Brooks gives the example of the *Comanche*, a “self-ferrying helicopter,” which was supposed to be able to fly itself over the Atlantic. This feature was not intended to be used frequently, yet it substantially complicated the design. Specifying features that are unlikely to be used, even those that are not difficult to implement, is sometimes derisively called gold-plating.

Obsolete Requirements

An *obsolete requirement* is one that has changed significantly (during system development) or no longer applies to the new system under consideration. For second, third, and so on generation systems and for systems derived from related products or in product lines (see Exercise 1.14), it is common practice to reuse portions of requirements specifications. When requirements specifications are reused, it is often the case that obsolete requirements are propagated. Whuk et al. (2013)

reported survey data from 219 respondents concerning obsolete requirements. They found that obsolete requirements are a significant problem for software-intensive products and in large projects. They also found that most of the companies surveyed did not have processes or automated tools for handling obsolete software requirements—only 10% of those surveyed reported using tools to support the identification of obsolete requirements.

Dubious requirements are related to obsolete requirements. Dubious requirements are those whose origin is unknown and/or which serve an unknown purpose. Thus, a dubious requirement may not be applicable in the current system, and thus, obsolete.

Obsolete and dubious requirements must be aggressively identified and culled through all phases of the requirements engineering lifecycle. Maintaining traceability links for requirements to their source and to related requirements is one significant way to prevent obsolete and dubious requirements. Date stamping requirements (upon their creation and for any changes) can help prevent obsolete requirements. Another, more labor-intensive way to manage obsolete and dubious requirements is by holding regular requirements inspections/reviews (discussed in Chapter 6).

Requirements churn refers to the changes in requirements that occur after the requirements are agreed upon. Requirements churn can be measured as the ratio of requirements changed per unit of time or as the ratio of requirements changed per total number of requirements at a given time. Requirements will change throughout the lifecycle of a system, but these changes have to be managed carefully. Maintaining historical statistics for the churn rate for projects at various stages and monitoring these rates can help to identify problems, such as too rapidly changing requirements. A high churn rate can be a leading indicator of obsolete requirements.

Four Dark Corners

Many of the problems with traditional requirements engineering arise from “four dark corners” (Zave and Jackson 1997). We repeat the salient points here, verbatim, with commentary in italics.

1. All the terminology used in requirements engineering should be grounded in the reality of the environment for which a machine is to be built.
2. It is not necessary or desirable to describe (however abstractly) the machine to be built.

Rather, the environment is described in two ways as it would be without, or in spite of, the machine and as we hope it will become because of the machine.

*Specifications are the **what** to be achieved by the system, not the **how**.*

3. Assuming that formal descriptions focus on actions, it is essential to identify which actions are controlled by the environment, which actions are controlled by the machine, and which actions of the environment are shared with the machine.

All types of actions are relevant to requirements engineering, and they might need to be described or constrained formally.

If formal descriptions focus on states, then the same basic principles apply in a slightly different form.

The method of formal representation should follow the underlying organization of the system.

For example, a state-based system is best represented by a state-based formalization.

4. The primary role of domain knowledge in requirements engineering is in supporting the refinement of requirements to implementable specifications.

Correct specifications, in conjunction with appropriate domain knowledge, imply the satisfaction of the requirements.

Failure to recognize the role of domain knowledge can lead to unfilled requirements and forbidden behavior.

Managing these dark corners is an important duty for requirements engineers and project managers.

Difficulties in Enveloping System Behavior

Imagine an arbitrary system with n inputs and m outputs.

Imagine that this system operates in an environment where the set of inputs, I , derives from human operators, sensors, storage devices, other systems, and so on. The outputs, O , pertain to display devices, actuators, storage devices, other systems, and so on. The only constraint we will place on the system is that the inputs and outputs are digitally represented within the system (if they are from/ to analog devices or systems, an appropriate conversion is needed). We define a behavior of the system as an input/output pair. Since the inputs and outputs are discrete, this system can be thought of as having an infinite but countable number of behaviors, $B \subseteq I \times O$.

Imagine the behavior space, B , is represented by the Venn diagram of Figure 1.2.

The leftmost circle in the diagram represents the desired behavior set, as it is understood by the customer. The area outside this circle is unwanted behavior and desirable behavior that, for whatever reason, the customer has not discovered.

The requirements engineer goes about his business and produces a specification that is intended to be representative of the behavior desired by the customer (the rightmost circle of Figure 1.2). Being imperfect, this specification captures some (but not all) of the desired behavior, and it also captures some undesirable behavior. The desired behavior not captured in the specified behavior is the missing functionality. The undesirable behavior that is captured is forbidden functionality.

The goal of the requirements engineer, in summary, is to make the left and right circles overlap as much as possible and to discover those missing desirable behaviors that are not initially encompassed by the specification (leftmost circle).

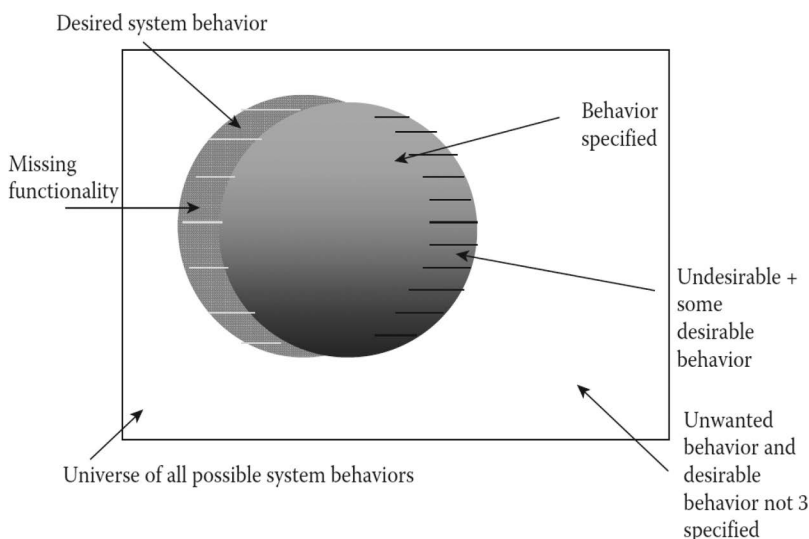


Figure 1.2 Universe of all possible behaviors, desired behavior, and specified behavior.

Recognizing that there is an infinite number of behaviors that could be excluded from the desired behaviors of the system, we need to, nevertheless, focus on the very worst of these. Therefore, some of the requirements that we will be discovering and communicating will take the form

The “system shall not ...”

A requirement that takes this form is sometimes called “unwanted functionality,” “shall not behavior,” or “forbidden behavior,” and it can be used to address a “hazard” situation. The term “hazard” is used to address something (an incident) that can potentially occur while the system is running (at run time), which can lead to failure and then consequently poses a danger or risk. For spaceborne communications satellites, for example, “Loss of communications with the Earth station” is an example of a hazard. “Power Failure” is another example of a hazard too.

On the other hand, “Project cost overruns” is not usually considered as a hazard because it is not being realized at run time, but at development time (while the system is being developed). In addition, by the same definition, requirements change, or standards changes are not considered as hazards.

A hazard by itself is not a requirement. For example, “power failure” is a hazard but not a requirement. But when we write a requirement to address hazards, we often write these in the form of “The system shall not...” statements. It is also possible to address hazards in the affirmative requirement form. For example, “The system shall not lose power” is equivalent in form to “The system shall provide continuous power.” Whether we chose to address the hazard with a requirement in the “shall not” or “shall” form is a matter of style and clarity.

On a snowy January 13 in 1982, an Air Florida Boeing 737 jet departed from Washington DC’s National Airport and crashed into the 14th Street Bridge less than a mile from the airport. Of the 79 people on board the plane, only 5 survived. The plane also struck seven vehicles on the bridge, killing four motorists and injuring another four.

The unfortunate passengers on the plane should have known that there was a risk of the plane crashing, no matter how small that risk may have been. In some cases, the passengers may have even taken out special crash insurance. But what were the expectations of the motorists in the vehicles on the bridge who were killed or injured? Surely, the thought of an airplane crashing into the bridge never crossed their mind. For the motorists, therefore, this crash represented an unknown but manifested uncertainty.

As an example of forbidden behavior, consider the following for the baggage handling system:

When the “conveyor jam” signal is set to the high state, the feed mechanism shall not permit additional items to enter the conveyor system.

We will discuss how to identify and characterize these unwanted or “shall not” behaviors in Chapter 3.

The Danger of “All” in Specifications

Requirement specification sentences often involve some universal quantification (e.g., “all users shall be able to access ...”). But the use of “all” specifications is dangerous because they are usually not true (Berry and Kamsties 2000). For example, if a requirement for the baggage handling system states: *All baggage shall be assigned a unique identifier.*

It is worthwhile to challenge this requirement. That is, is there some form of “baggage” that we wish to not give a unique identifier? If a suspicious package is given to an airport check-in agent, should they print out a barcode tag and put it on the package? Or, should they put it aside for

further processing, such as inspection by security personnel? Whether the answer is “yes” or “no” is irrelevant—we want to consider these kinds of situations and deal with them as we are writing requirements in draft form, rather than postpone the issue until requirements analysis occurs. Therefore, requirements involving the word “all” should be challenged and relaxed when possible.

Related to “all” specifications are “never,” “always,” “none,” and “each” (because they can be formally equated to universal quantification), these words and their mathematical equivalents should be avoided as well.

VIGNETTE 1.2 Government Procurement

In the United States, the requirements engineering process required of a contractor providing some system to a government entity is quite rigorous. The government entity, such as the Department of Defense or Department of Energy, will often solicit project proposals based on an already well-defined set of requirements. For example, many of the requirements for a new nuclear power generation plant will be based on previous systems, which, in turn, are largely based on standards, regulations, and laws. These requirements are established by the contracting agency (and other stakeholders) before the work contract is awarded.

Contractors can participate in the contract proposal generation through industry days. During an industry day, the government entity will convene potential contractors to discuss the current requirements of the system. This process is iterative, and as the prebid requirements become more mature, the system becomes more well-defined.

After the contract is awarded, the requirements engineering process continues. Throughout, the winning contractor will use many of the techniques described in the book to refine system requirements. It is essential that during this process contractor customer communications are well structured. Therefore, some of the more informal approaches to requirements engineering will be prohibited (and often the contract states which methodologies can or cannot be used). For example, holding a number of JAD sessions (discussed in Chapter 3) is often a contractual requirement.

The process described is similar to government contracting in many countries.

Exercises

- 1.1 What are some of the major objections and deterrents to proper requirements engineering activities?
- 1.2 How is requirements engineering different for “small” systems?
- 1.3 How is requirements engineering different for “startup” businesses?
- 1.4 Emerging technologies including AI (artificial intelligence), 5G, IoT (Internet of Things), serverless computing, biometrics, AR (augmented reality)/VR (virtual reality), edge computing, blockchain, robotics, NLP (natural language processing), and quantum computing, whose development, practical applications, or both are still largely unrealized. How is requirements engineering different when developing “Emerging Technologies” based systems?

- 1.5 What are some factors that may cause customers to alter requirements?
- 1.6 What issues might arise when a requirements engineer, who is not a subject matter expert, enlists a subject matter expert to assist in defining requirements?
- 1.7 List some representative user requirements, system requirements, and software specifications for
 - 1.7.1 The pet store POS system
 - 1.7.2 The baggage handling system
- 1.8 List five typical functional requirements for
 - 1.8.1 The pet store POS system
 - 1.8.2 The baggage handling system
- 1.9 List five forbidden functional requirements for
 - 1.9.1 The pet store POS system
 - 1.9.2 The baggage handling system
- 1.10 Identify the nonfunctional requirements in
 - 1.10.1 The specification for the smart home system in Appendix A
 - 1.10.2 The specification for the wet-well pumping system in Appendix B
- 1.11 List five possible nonfunctional requirements for
 - 1.11.1 The pet store POS system
 - 1.11.2 The baggage handling system
- 1.12 Conduct some Web research to discover prevailing regulations or standards (NFR) for smart home systems.
- 1.13 For the smart home system make a list of some of the hazards (what this system shall not do) based on the regulations you discovered in Exercise 1.12 and any other information you might have.
- *1.14 A product line is a set of related products that share certain features. Product lines may be planned or they may emerge. What are some of the requirements engineering challenges for product lines? How does the “one-shot solution” aspect of Ritter and Webber’s wicked problem apply? You may wish to conduct some research to answer this question.

References

- ACM/IEEE. (2014). Computer Society Curriculum Guidelines for Undergraduate Degree Programs in Software Engineering. <https://www.acm.org/education/se2014.pdf> (accessed January 2017).
- Al-Ani, B., & Sim, S. E. (2006). So, you think you are a requirements engineer? In *14th IEEE International Requirements Engineering Conference (RE'06)*, Minneapolis/St. Paul, MN, USA (pp. 337–338). IEEE.
- Berry, D. M. (1995). The importance of ignorance in requirements engineering. *Journal of Systems and Software*, 28(1): 179–184.
- Berry, D. M. (1999). Software and house requirements engineering: Lessons learned in combating requirements creep. *Requirements Engineering Journal*, 3(3&4): 242–244.
- Berry, D. M. (2003). More requirements engineering adventures with building contractors. *Requirements Engineering Journal*, 8(2): 142–146.
- Berry, D. M., & Kamsties, E. (2000). The dangerous ‘All’ in specifications. In *Proceedings of the Tenth International Workshop on Software Specification and Design (IWSSD'00)*, San Diego, CA, 5–7 November.
- Brooks, F. P., Jr. (2010). *The Design of Design: Essays from a Computer Scientist*. Pearson Education, Addison-Wesley Professional, Boston, MA.

- Calazans, A. T. S., Paldès, R. Á., Masson, E. T. S., Brito, I. S., Rezende, K. F., Braosi, E., & Pereira, N. (2017). Software requirements analyst profile: A descriptive study of Brazil and Mexico. In *2017 IEEE 25th International Requirements Engineering Conference (RE)*, Lisbon, Portugal (pp. 204–212). IEEE.
- Christensen, M., and Chang, C. (1996). Blueprint for the ideal requirements engineer. *IEEE Software*, 13(2): 12.
- Daneva, M., Wang, C., & Hoener, P. (2017). What the job market wants from requirements engineers? An empirical analysis of online job ads from the Netherlands. In *2017 ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM)* (pp. 448–453). IEEE.
- Ebert, C. (2010). Requirements engineering: Management. In P. Laplante (Ed.), *Encyclopedia of Software Engineering* (pp. 932–948). Taylor & Francis, Boca Raton, FL.
- Gorla, N., & Lam, Y. W. (2004). Who should work with whom? Building effective software project teams. *Communications of the ACM*, 47(6): 79–82.
- Graduate Reference Curriculum for Systems Engineering (GRCSE). (2012). <http://www.bkcase.org/grcse/> (accessed January 2017).
- Graduate Software Engineering Reference Curriculum (GSWE). (2009). <https://www.acm.org/binaries/content/assets/education/gsew2009.pf> (accessed January 2017).
- Guide to the Systems Engineering Body of Knowledge (SEBoK). (2012). <http://www.sebokwiki.org/> (accessed January 2017).
- Herrmann, A. (2013). Requirements engineering in practice: There is no requirements engineer position. In *International Working Conference on Requirements Engineering: Foundation for Software Quality* (pp. 347–361). Springer, Berlin, Heidelberg.
- The Information and Communications Technology Council. The Smartest Economy Reshaping Canada's Workforce: Labor Market Outlook 2015–2019.
- Institute for Business Analysis. (2010). A Guide to the Business Analysis Body of Knowledge (BABOK Guide), Version 3.
- Kanaracus, C. (2008). Study: Bad Requirements-Gathering Hurts IT Projects. NetworkWorld.com.
- Kassab, M. (2021, September). How requirements engineering is performed in small businesses? In *2021 IEEE 29th International Requirements Engineering Conference Workshops (REW)* (pp. 220–223). IEEE.
- Kassab, M., & Laplante, P. (2022). The current and evolving landscape of requirements engineering in practice. *IEEE Software*. DOI: 10.1109/MS.2022.3147692.
- Kassab, M., Neill, C., & Laplante, P. (2014). State of practice in requirements engineering: Contemporary data. *Innovations in Systems and Software Engineering*, 10(4): 235–241.
- Kilcay-Ergin, N., & Laplante, P. A. (2013). An online graduate requirements engineering course. *IEEE Transactions on Education*, 56(2): 199–207.
- Klendauer, R., Berkovich, M., Gelvin, R., Leimeister, J. M., & Krcmar, H. (2012). Towards a competency model for requirements analysts. *Information Systems Journal*, 22(6): 475–503.
- Laplante, P. A., Kalinowski, B., & Thornton, M. (2013). A principles and practices exam specification to support software engineering licensure in the United States of America. *Software Quality Professional*, 15(1): 4–15.
- Penzenstadler, B., Fernández, D. M., Richardson, D., Callele, D., & Wnuk, K. (2013). The requirements engineering body of knowledge (rebok). In *2013 21st IEEE International Requirements Engineering Conference (RE)*, Rio de Janeiro, Brazil (pp. 377–379). IEEE.
- Project Management Institute. (2014). PMI: A Guide to the Project Management Body of Knowledge, Version 4.
- Rittel, H., & Webber, M. (1973). Dilemmas in a general theory of planning. *Policy Sciences*, 4: 155–169.
- Royce, W. (1975). *Practical Strategies for Developing Large Software Systems*. Addison-Wesley, Boston, MA, p. 59.
- Sikora, E., Tenbergen, B., & Pohl, K. (2012). Industry needs and research directions in requirements engineering for embedded systems. *Requirements Engineering*, 17: 57–78.
- Software Engineering Body of Knowledge Version 3.0 (SWEBOK). (2014). <http://www.computer.org/portal/web/swebok/html/contentsch2#ch2> (accessed January 2017).
- Sommerville, I. (2005). *Software Engineering*, 7th edition. Addison-Wesley, Boston, MA.
- US Department of Labor: Occupational Outlook Handbook. (2015).

- Wang, C., Cui, P., Daneva, M., & Kassab, M. (2018). Understanding what industry wants from requirements engineers: An exploration of RE jobs in Canada. In *Proceedings of the 12th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement* (pp. 1–10), Oulu, Finland.
- Wnuk, K., Gorschek, T., & Zahda, S. (2013). Obsolete software requirements. *Information and Software Technology*, 55(6): 921–940.
- Wnuk, K., Regnell, B., & Berenbach, B. (2011). Scaling up requirements engineering—Exploring the challenges of increasing size and complexity in market-driven software development. In D. Berry & X. French (Eds.), *Requirements Engineering: Foundation for Software Quality. REFSQ 2011*, LNCS (Vol. 6606, pp. 54–59), Essen, Germany.
- Zachman, J. A. (1987). A framework for information systems architecture. *IBM Systems Journal*, 26(3): 276–292.
- Zave, P. (1997). Classification of research efforts in requirements engineering. *ACM Computing Surveys*, 29(4): 315–321.
- Zave, P., and Jackson, M. (1997). Four dark corners of requirements engineering. *ACM Transactions on Software Engineering Methodology*, 6(1): 1–30.

Introduction to Requirements Engineering

- ACM/IEEE . (2014). Computer Society Curriculum Guidelines for Undergraduate Degree Programs in Software Engineering. <https://www.acm.org/education/se2014.pdf> (accessed January 2017).
- Al-Ani, B. , & Sim, S. E. (2006). So, you think you are a requirements engineer? In 14th IEEE International Requirements Engineering Conference (RE'06), Minneapolis/St. Paul, MN, USA (pp. 337–338). IEEE.
- Berry, D. M. (1995). The importance of ignorance in requirements engineering. *Journal of Systems and Software*, 28(1): 179–184.
- Berry, D. M. (1999). Software and house requirements engineering: Lessons learned in combating requirements creep. *Requirements Engineering Journal*, 3(3&4): 242–244.
- Berry, D. M. (2003). More requirements engineering adventures with building contractors. *Requirements Engineering Journal*, 8(2): 142–146.
- Berry, D. M. , & Kamsties, E. (2000). The dangerous 'All' in specifications. In *Proceedings of the Tenth International Workshop on Software Specification and Design (IWSSD'00)*, San Diego, CA, 5–7 November.
- Brooks, F. P. Jr. , (2010). *The Design of Design: Essays from a Computer Scientist*. Pearson Education, Addison-Wesley Professional, Boston, MA.
- Calazans, A. T. S. , Paldês, R. Á. , Masson, E. T. S. , Brito, I. S. , Rezende, K. F. , Braosi, E. , & Pereira, N. (2017). Software requirements analyst profile: A descriptive study of Brazil and Mexico. In 2017 IEEE 25th International Requirements Engineering Conference (RE), Lisbon, Portugal (pp. 204–212). IEEE.
- Christensen, M. , and Chang, C. (1996). Blueprint for the ideal requirements engineer. *IEEE Software*, 13(2): 12.
- Daneva, M. , Wang, C. , & Hoener, P. (2017). What the job market wants from requirements engineers? An empirical analysis of online job ads from the Netherlands. In 2017 ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM) (pp. 448–453). IEEE.
- Ebert, C. (2010). Requirements engineering: Management. In P. Laplante (Ed.), *Encyclopedia of Software Engineering* (pp. 932–948). Taylor & Francis, Boca Raton, FL.
- Gorla, N. , & Lam, Y. W. (2004). Who should work with whom? Building effective software project teams. *Communications of the ACM*, 47(6): 79–82.
- Graduate Reference Curriculum for Systems Engineering (GRCSE) . (2012). <http://www.bkcase.org/grcse/> (accessed January 2017).
- Graduate Software Engineering Reference Curriculum (GSwE) . (2009). <https://www.acm.org/binaries/content/assets/education/gsew2009.pdf> (accessed January 2017).
- Guide to the Systems Engineering Body of Knowledge (SEBoK) . (2012). <http://www.sebokwiki.org/> (accessed January 2017).
- Herrmann, A. (2013). Requirements engineering in practice: There is no requirements engineer position. In *International Working Conference on Requirements Engineering: Foundation for Software Quality* (pp. 347–361). Springer, Berlin, Heidelberg.
- The Information and Communications Technology Council . *The Smartest Economy Reshaping Canada's Workforce: Labor Market Outlook 2015–2019*.
- Institute for Business Analysis . (2010). *A Guide to the Business Analysis Body of Knowledge (BABOK Guide), Version 3*.
- Kanaracus, C. (2008). Study: Bad Requirements-Gathering Hurts IT Projects. *NetworkWorld.com*.
- Kassab, M. (2021, September). How requirements engineering is performed in small businesses? In 2021 IEEE 29th International Requirements Engineering Conference Workshops (REW) (pp. 220–223). IEEE.
- Kassab, M. , & Laplante, P. (2022). The current and evolving landscape of requirements engineering in practice. *IEEE Software*. DOI: 10.1109/MS.2022.3147692.
- Kassab, M. , Neill, C. , & Laplante, P. (2014). State of practice in requirements engineering: Contemporary data. *Innovations in Systems and Software Engineering*, 10(4): 235–241.
- Kilcay-Ergin, N. , & Laplante, P. A. (2013). An online graduate requirements engineering course. *IEEE Transactions on Education*, 56(2): 199–207.
- Klendauer, R. , Berkovich, M. , Gelvin, R. , Leimeister, J. M. , & Krcmar, H. (2012). Towards a competency model for requirements analysts. *Information Systems Journal*, 22(6): 475–503.
- Laplante, P. A. , Kalinowski, B. , & Thornton, M. (2013). A principles and practices exam specification to support software engineering licensure in the United States of America. *Software Quality Professional*, 15(1): 4–15.
- Penzenstadler, B. , Fernández, D. M. , Richardson, D. , Callele, D. , & Wnuk, K. (2013). The requirements engineering body of knowledge (rebok). In 2013 21st IEEE International Requirements Engineering Conference (RE), Rio de Janeiro, Brazil (pp. 377–379). IEEE.
- Project Management Institute . (2014). *PMI: A Guide to the Project Management Body of Knowledge, Version 4*.
- Rittel, H. , & Webber, M. (1973). Dilemmas in a general theory of planning. *Policy Sciences*, 4: 155–169.
- Royce, W. (1975). *Practical Strategies for Developing Large Software Systems*. Addison-Wesley, Boston, MA, p. 59.

Sikora, E. , Tenbergen, B. , & Pohl, K. (2012). Industry needs and research directions in requirements engineering for embedded systems. *Requirements Engineering*, 17: 57–78.

Software Engineering Body of Knowledge Version 3.0 (SWEBOOK) . (2014).
<http://www.computer.org/portal/web/swebok/html/contentsch2#ch2> (accessed January 2017).

Sommerville, I. (2005). *Software Engineering*, 7th edition. Addison-Wesley, Boston, MA.

US Department of Labor: Occupational Outlook Handbook . (2015).

Wang, C. , Cui, P. , Daneva, M. , & Kassab, M. (2018). Understanding what industry wants from requirements engineers: An exploration of RE jobs in Canada. In *Proceedings of the 12th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement* (pp. 1–10), Oulu, Finland.

Wnuk, K. , Gorschek, T. , & Zahda, S. (2013). Obsolete software requirements. *Information and Software Technology*, 55(6): 921–940.

Wnuk, K. , Regnell, B. , & Berenbach, B. (2011). Scaling up requirements engineering—Exploring the challenges of increasing size and complexity in market-driven software development. In D. Berry & X. French (Eds.), *Requirements Engineering: Foundation for Software Quality. REFSQ 2011, LNCS* (Vol. 6606, pp. 54–59), Essen, Germany.

Zachman, J. A. (1987). A framework for information systems architecture. *IBM Systems Journal*, 26(3): 276–292.

Zave, P. (1997). Classification of research efforts in requirements engineering. *ACM Computing Surveys*, 29(4): 315–321.

Zave, P. , and Jackson, M. (1997). Four dark corners of requirements engineering. *ACM Transactions on Software Engineering Methodology*, 6(1): 1–30.

Preparing for Requirements Elicitation

Asnar, Y. , Giorgini, P. , & Mylopoulos, J. (2011). Goal-driven risk assessment in requirements engineering. *Requirements Engineering*, 16(2): 101–116.

Boehm, B. (2003). *Balancing, Agility and Discipline, A Guide for the Perplexed*. Addison-Wesley, Boston, MA.

Checkland, P. , & Scholes, J. (1999). *Soft Systems Methodology in Action*. Wiley, West Sussex.

Cohen, H. (2000). *You Can Negotiate Anything*. Citadel, New York.

Crane, A. , & Ruebottom, T. (2011). Stakeholder theory and social identity: Rethinking stakeholder identification. *Journal of Business Ethics*, 102(1): 77–87.

Damian, D. (2007). Stakeholders in global requirements engineering: Lessons learned from practice. *IEEE Software*, 24(2): 21–27.

Felfernig, A. , Zehentner, C. , Ninaus, G. , Grabner, H. , Maalej, W. , Pagano, D. , ... Reinfrank, F. (2011, July). Group decision support for requirements negotiation. In *International Conference on User Modeling, Adaptation, and Personalization* (pp. 105–116). Springer, Berlin, Heidelberg.

Glinz, M. , & Wieringa, R. J. (2007). Guest editors' introduction: Stakeholders in requirements engineering. *IEEE Software*, 24(2): 18–20.

Gregory, A. J. , Atkins, J. P. , Midgley, G. , & Hodgson, A. M. (2020). Stakeholder identification and engagement in problem structuring interventions. *European Journal of Operational Research*, 283(1): 321–340.

Hofstede, G. (2001). *Culture's Consequences: Comparing Values, Behaviors, Institutions and Organizations across Nations*. Sage, Thousand Oaks, CA.

Hull, E. , Jackson, K. , & Dick, J. (2011). *Requirements Engineering in the Problem Domain*, *Requirements Engineering*, 2nd edition. Springer-Verlag, London, pp. 93–114.

Hunter, J. (2015). The hierarchy of IoT "Thing" needs. *Techcrunch*, 15 September, 2015 .
<https://techcrunch.com/2015/09/05/the-hierarchy-of-iot-thing-needs/> (accessed November 2016).

Jeon, C. K. , Kim, N. H. , Lee, D. H. , Lee, T. , & In, H. P. (2012). Stakeholder Conflict Resolution Model (S-CRM) based on supervised learning. *KSII Transactions on Internet and Information Systems (TIIS)*, 6(11): 2813–2826.

Khan, U. Z. , Wahab, F. , & Saeed, S. (2014). Integration of scrum with win-win requirements negotiation model. *Middle-East Journal of Scientific Research*, 19(1): 101–104.

Laplane, N. L. , Laplane, P. A. , & Voas, J. M. (2016). Stakeholder identification and use case representation for Internet-of-things applications in healthcare. *IEEE Systems Journal*. DOI: 10.1109/JSYST.2016.2558449.

Laplane, P. A. (2010). Where in the world is Carmen Sandiego (and is she a software engineer)? *IT Professional*, 12(6): 10–13.

Li, T. , Zhang, H. , Liu, Z. , Ke, Q. , & Alting, L. (2014). A system boundary identification method for life cycle assessment. *The International Journal of Life Cycle Assessment*, 19(3): 646–660.

Mairiza, D. , Zowghi, D. , & Gervasi, V. (2014). Utilizing topsis: A multi criteria decision analysis technique for non-functional requirements conflicts. In *Requirements Engineering* (pp. 31–44). Springer, Berlin, Heidelberg.

Maslow, A. (1943). A theory of human motivation. *Psychological Review*, 50: 370–396.

Newell, A. F. , Dickinson, A. , Smith, M. J. , & Gregor, P. (2006). Designing a portal for older users: A case study of an industrial/academic collaboration. *ACM Transactions on Computer-Human Interaction*, 13(3): 347–375.

Narendhar, M. , & Anuradha, K. (2017). Requirement negotiation methods in requirements engineering. *International Journal of Advanced Research in Computer Science*, 8(3): 1–6.

Valacich, J. H. , Parboteeah, D. V. , & Wells, J. D. (2007). The online consumer's hierarchy of needs. *Communications of the ACM*, 50(9): 84–90.

Wang, W. , Liu, W. , & Mingers, J. (2015). A systemic method for organizational stakeholder identification and analysis using Soft Systems Methodology (SSM). *European Journal of Operational Research*, 246(2): 562–574.

Requirements Elicitation

Adepetu, A. , Khaja, A. A. , Al Abd , Y. , Al Zaabi , A. , & Svetinovic, D. (2012, March). CrowdREquire: A requirements engineering crowdsourcing platform. In 2012 AAAI Spring Symposium Series, Palo Alto, California.

Akao, Y. (1990). *Quality Function Deployment: Integrating Customer Requirements into Product Design*. Productivity Press, Cambridge, MA.

Alderson, A. , & Liu, K. (2012). Reverse requirements engineering: The AMBOLS approach. *Systems Engineering for Business Process Change: Collected Papers from the EPSRC Research Programme* (pp. 196–208). Springer, London.

Basili, V. R. , & Weiss, D. (1984). A methodology for collecting valid software engineering data. *IEEE Transactions on Software Engineering*, 10: 728–738.

Berman, B. (2012). 3-D printing: The new industrial revolution. *Business Horizons*, 55(2): 155–162.

Borg, A. , Yong, A. , Carlshamre, P. , & Sandahl, K. (2003). The bad conscience of requirements engineering: An investigation in real-world treatment of non-functional requirements. In *Third Conference on Software Engineering Research and Practice in Sweden (SERPS'03)* (pp. 1–8). Lund.

Centers for Medicare & Medicaid Services . (1996). The Health Insurance Portability and Accountability Act of 1996 (HIPAA). Online at <http://www.hhs.gov/hipaa> (accessed June 2017).

Cleland-Huang, J. (2014). How well do you know your personae non gratae? *IEEE Software*, 31(4): 28–31.

Cleland-Huang, J. , Denning, T. , Kohno, T. , Shull, F. , & Weber, S. (2016). Keeping ahead of our adversaries. *IEEE Software*, 33(3): 24–28.

Cleland-Huang, J. , & Laurent, P. (2014). Requirements in a global world. *IEEE Software*, 31(6): 34–37.

Eberlein, A. , & Jiang, L. (2011). Selecting requirements engineering techniques. In P. Laplante (Ed.), *Encyclopedia of Software Engineering* (pp. 962–978). Taylor & Francis. Boca Raton, FL, Published online.

Fahmi, S. A. , & Choi, H. J. (2007, November). Software reverse engineering to requirements. In 2007 International Conference on Convergence Information Technology (ICCIT 2007), (pp. 2199–2204). IEEE, Gwangju, South Korea.

Groen, E. C. , Doerr, J. , & Adam, S. (2015, March). Towards crowd-based requirements engineering a research preview. In *International Working Conference on Requirements Engineering: Foundation for Software Quality* (pp. 247–253). Springer, Cham.

Groen, E. C. , & Koch, M. (2016). How requirements engineering can benefit from crowds. *Requirements Engineering Magazine*, 8: 10.

Hassan, S. , Qamar, U. , Hassan, T. , & Waqas, M. (2015, August). Software reverse engineering to requirement engineering for evolution of legacy system. In 2015 5th International Conference on IT Convergence and Security (ICITCS) (pp. 1–4). IEEE, Kuala Lumpur, Malaysia.

Herrmann, A. , Kerkow, D. , & Doerr, J. (2007). Exploring the characteristics of NFR methods – A dialogue about two approaches. In *REFSQ 2007, LNCS 4542* (pp. 320–334). Trondheim, Norway.

Hickey, A. M. , & Davis, A. M. (2003). Elicitation technique selection: How do experts do it? In *Proceedings 11th IEEE International Requirements Engineering Conference, 2003* (pp. 169–178). IEEE, Monterey Bay, CA, USA.

Howe, J. (2006). The rise of crowdsourcing. *Wired Magazine*, 14(6): 1–4.

Issa, A. A. , & Al-Ali, A. (2010). Use case patterns driven requirements engineering. In *Proceedings Second International Conference on Computer Research and Development* (pp. 307–313). Kuala Lumpur, Malaysia.

Kassab, M. , & Laplante, P. (2022). The current and evolving landscape of requirements engineering state of practice. *IEEE Software*. DOI: 10.1109/MS.2022.3147692.

Kassab, M. , Neill, C. , & Laplante, P. (2014). State of practice in requirements engineering: Contemporary data. *Innovations in Systems and Software Engineering*, 10(4): 235–241.

Kassab, M. , & Ormandjieva, O. (2014). Non-functional requirements in process-oriented approaches. In Phillip A. Laplante (Ed.), *Encyclopedia of Software Engineering* (pp. 1–11). Taylor & Francis, Boca Raton, FL.

Laplante, P.A. (2006). *What Every Engineer Needs to Know About Software Engineering*. CRC/Taylor & Francis, Boca Raton, FL.

Lim, S. L. , & Finkelstein, A. (2011). StakeRare: Using social networks and collaborative filtering for large-scale requirements elicitation. *IEEE Transactions on Software Engineering*, 38(3): 707–735.

NASA (National Aeronautics and Space Administration) . (2013). NASA-STD 8719.13, Software Safety Standard. <http://NASA.gov> (accessed January 2017).

Renzel, D. , & Klamma, R. (2014). Requirements bazaar: Open-source large scale social requirements engineering in the long tail. *IEEE Computer Society Special Technical Community on Social Networking E-Letter*, Vol. 2 No. 3.

Robinson, W. N. (2010). A roadmap for comprehensive requirements modeling. *Computer*, 43(5): 64–72.

Snijders, R. , Dalpiaz, F. , Brinkkemper, S. , Hosseini, M. , Ali, R. , & Ozum, A. (2015, August). REfine: A gamified platform for participatory requirements engineering. In *2015 IEEE 1st International Workshop on Crowd-Based Requirements Engineering (CrowdRE)* (pp. 1–6). IEEE, Ottawa, Canada.

Sommerville, I. , & Sawyer, P. (1997). Viewpoints for requirements engineering. *Software Quality Journal*, 3: 101–130.

Voas, J. , & Laplante, P. (2010). Effectively defining shall not requirements. *IT Professional*, 12(3): 46–53.

Yu, Y. , Wang, Y. , Mylopoulos, J. , Liaskos, S. , Lapouchnian, A. , & do Prado Leite, J. C. S. (2005, September). Reverse engineering goal models from legacy code. In *13th IEEE International Conference on Requirements Engineering (RE'05)* (pp. 363–372). IEEE, Paris, France.

Zowghi, D. , & Coulin, C. (1998). Requirements elicitation: A survey of techniques, approaches, and tools. In A. Aurum & C. Wohlin (Eds.), *Engineering and Managing Software Requirements* (pp. 19–46). Springer, Berlin.

Writing the Requirements Document

Ron Ross , Victoria Pillitteri , Gary Guissanie , Ryan Wagner , Richard Graubart and Deborah Bodeau , NIST SP 800-172 Enhanced Security Requirements for Protecting Controlled Unclassified Information: A Supplement to NIST Special Publication 800-171, February 2021, available at <https://csrc.nist.gov/publications/detail/sp/800-172/final>

Berenbach, B. , Schneider, F. , & Naughton, H. (2012, September). The use of a requirements modeling language for industrial applications. In *2012 20th IEEE International Requirements Engineering Conference (RE)* (pp. 285–290). IEEE, Chicago, IL, USA.

Dardenne, A. , Van Lamsweerde, A. , & Fickas, S. (1993). Goal-directed requirements acquisition. *Science of Computer Programming*, 20(1–2): 3–50.

Dassault Systems . (2016). CATIA Reqify. Retrieved from <https://www.3ds.com/products-services/catia/products/reqify>

Dick, J. (2010). Requirements engineering: Principles and practice. In P. Laplante (Ed.), *Encyclopedia of Software Engineering* (pp. 949–961). Taylor & Francis, Boca Raton, FL, Published online.

Eberlein, A. , & Jiang, L. (2011). Requirements engineering: Technique selection. In P. Laplante (Ed.), *Encyclopedia of Software Engineering* (pp. 962–978). Taylor & Francis, Boca Raton, FL, Published online.

Elcock, A. , & Laplante, P. A. (2006). Testing without requirements. *Innovations in Systems and Software Engineering: A NASA Journal*, 2: 137–145.

Gilb, T. (2005). *Competitive Engineering: A Handbook for Systems Engineering, Requirements Engineering, and Software Engineering Using Planguage*. 1 edition. Butterworth-Heinemann, UK.

Glinz, M. (2008). A risk-based, value-oriented approach to quality requirements. *Computer*, 25(2): 34–41.

Hammer, T. F. , Huffman, L. L. , Rosenberg, L. H. , Wilson, W. , & Hyatt, L. (1998). Doing requirements right the first time. *CROSSTALK The Journal of Defense Software Engineering*, 1: 20–25.

Hay, D. (2003). *Requirements Analysis: From Business Views to Architecture*. Prentice Hall PTR, Upper Saddle River, NJ.

Hull, E. , Jackson, K. , & Dick, J. (2011). Requirements engineering in the problem domain. In E. Hull , K. Jackson , & J. Dick . (Eds.), *Requirements Engineering* (pp. 93–114). Springer-Verlag, London.

IEEE Standard 830-1998 . (1998). *Recommend Practice for Software Requirements Specifications*. IEEE Standards Press, Piscataway, NJ.

ISO/IEC/IEEE Standard 29148 . (2011). *Systems and Software Engineering—Life Cycle Processes—Requirements Engineering*. IEEE, Piscataway, NJ.

Jeannet, B. , & Gaucher, F. (2015). Debugging real-time systems requirements: simulate the “what” before the “how”. In *Embedded World Conference, Nürnberg, Germany*.

Kassab, M. , & Laplante, P. (2022). The current and evolving landscape of requirements engineering state of practice. *IEEE Software*. DOI: 10.1109/MS.2022.3147692.

Laplante, P. A. (2006). *What Every Engineer Should Know about Software Engineering*. CRC Press, Boca Raton, FL.

Laplante, P. A. (2019). *Technical Writing: A Practical Guide for Engineers and Scientists*, 2nd edition. CRC Press, Taylor & Francis, Boca Raton, FL.

Meyer, B. , Bruel, J. M. , Ebersold, S. , Galinier, F. , & Naumchev, A. (2019, October). Towards an anatomy of software requirements. In *International Conference on Objects, Components, Models and Patterns* (pp. 10–40). Springer, Cham.

NASA WIRE (Wide-Field Infrared Explorer) Software Requirements Specification. (1996). <http://science.nasa.gov/missions/wire/> (accessed January 2017).

Neill, C. J. , & Laplante, P. A. (2003). Requirements engineering: The state of the practice. *Software*, 20(6): 40–45.

Norden, B. (2007). Screenwriting for requirements engineers. *Software*, 24: 26–27.

Ricca, F. , Scanniello, G. , Torchiano, M. , Reggio, G. , & Astesiano, E. (2010). On the effectiveness of screen mockups in requirements engineering: Results from an internal replication. In *Proceedings of the 2010 ACM-IIEEE International Symposium on Empirical Software Engineering and Measurement* (Vol. 17, p. 26), Bolzano, Italy.

Scott, W. , & Cook, S. C. (2004). A Context-Free Requirements Grammar to Facilitate Automatic Assessment, AWRE'04 9th Australian Workshop on Requirements Engineering, Adelaide, Australia.

Sutcliffe, A. , Thew, S. , & Jarvis, P. (2011). Experience with user-centred requirements engineering. *Requirements Engineering*, 16(4): 267–280.

Whittle, J. , Sawyer, P. , Bencomo, N. , Cheng, B. H. , & Bruel, J. M. (2009, August). Relax: Incorporating uncertainty into the specification of self-adaptive systems. In *2009 17th IEEE International Requirements Engineering Conference* (pp. 79–88). IEEE, Atlanta, GA, USA.

Wilson, W. M. , Rosenberg, L. H. , & Hyatt, L. E. (1997). Automated analysis of requirement specifications. In *Proceedings of the 19th International Conference on Software Engineering* (pp. 161–171). Boston, Massachusetts, USA.

On Nonfunctional Requirements

Antón, A. (1997). Goal Identification and Refinement in the Specification of Information Systems, Ph.D. Thesis, Georgia Institute of Technology, Georgia, USA.

Araujo, J. , Moreira, A. , Brito, I. , & Rashid, A. (2002). Aspect-oriented requirements with UML. Workshop on Aspect-Oriented Modeling with UML (Held with UML 2002, Dresden, Germany).

Babcock, C. (1985, September). New Jersey Motorists in Software Jam. *Computerworld*, 30: 1–6.

Barbacci, M. R. , Robert E. , Anthony J. L. , Judith A. S. , & Charles B. W. (2003). Quality attribute workshops (qaws). CARNEGIE-MELLON UNIV PITTSBURGH PA SOFTWARE ENGINEERING INST.

Bass, L. , Clements, P. , & Kazman, R. (2021). *Software Architecture in Practice*, 4th edition. Addison-Wesley Professional, Boston, MA, USA.

Boehm, B. , & In, H. (1996). Identifying quality-requirement conflicts. *IEEE Software*, IEEE Computer Society Press, pp. 25–35. Washington, D.C., United States.

Boehm, B. W. , Brown, J. R. , & Lipow, M. (1976). Quantitative evaluation of software quality. In *Proceeding of the 2nd International Conference on Software Engineering* (pp. 592–605). IEEE Computer Society, San Francisco, CA, Long Branch, CA.

Breitman, K. K. , Leite J. C. S. P. , & Finkelstein, A. (1999). The world's stage: A survey on requirements engineering using a real-life case study. *Journal of the Brazilian Computer Society*, 1(6): 13–37.

Chung, L. , Nixon, B. A. , Yu, E. , & Mylopoulos, J. (2000). *Nonfunctional Requirements in Software Engineering*. Springer, Boston, MA.

Davis, A. (1993). *Software Requirements: Objects, Functions and States*. Prentice Hall, Hoboken, New Jersey, United States.

Fenton, N. E. , & Pfleeger, S. L. (1997). *Software Metrics: A Rigorous and Practical Approach*. International Thomson Computer Press.

Finkelstein, A. , & Dowell, J. (1996). A comedy of errors: The London Ambulance Service case study. *Proceedings of the 8th International Workshop Software Specifications and Design* (pp. 2–5), Schloss Velen, Germany.

Gilb, T. (2005). *Competitive Engineering: A Handbook for Systems Engineering, Requirements Engineering, and Software Engineering Using Planguage*. Elsevier.

Grady, R. B. (1992). *Practical Software Metrics for Project Management and Process Improvement*. Prentice-Hall, Hoboken, New Jersey, United States.

ISO 25000 STANDARDS. <https://iso25000.com/index.php/en/iso-25000-standards>

Jacobson, I. , Booch, G. , & Rumbaugh, J. (1999). *The Unified Software Development Process*. Addison Wesley, Reading, MA.

Kassab, M. (2009). *Non-Functional Requirements: Modeling and Assessment*. VDM Verlag, Germany.

Kassab, M. , Daneva, M. , & Ormandjieva, O. (2007, August). Scope management of non-functional requirements. In *33rd EUROMICRO Conference on Software Engineering and Advanced Applications (EUROMICRO 2007)* (pp. 409–417). IEEE, Lubeck, Germany.

Kassab, M. , DeFranco, J. , & Laplante, P. (2020). A systematic literature review on Internet of things in education: Benefits and challenges. *Journal of Computer Assisted Learning*, 36(2): 115–127.

Kassab, M. , DeFranco, J. , & Voas, J. (2018). Smarter education. *IT Professional*, 20(5): 20–24.

Keller, S. E. , Kahn, L. G. , & Panara, R. B. (1990). Specifying software quality requirements with metrics. In R. H. Thayer & M. Dorfman (Eds.), *System and Software Requirements Engineering* (pp. 145–163). IEEE Computer Society Press, Washington, DC.

Kotonya, G. , & Sommerville, I. (1998). *Requirements Engineering: Processes and Techniques*. John Wiley & Sons, Hoboken, New Jersey, United States.

Laplante, P. A. , Kassab, M. , Laplante, N. L. , & Voas, J. M. (2017). Building caring healthcare systems in the Internet of Things. *IEEE Systems Journal*, 12(3): 3030–3037.

Leffingwell, D. , & Widrig, D. (2003). *Managing Software Requirements: A Unified Approach*. The Addison-Wesley Object Technology Series, Boston, Massachusetts, United States.

Lyu, M. R. (1996). *Handbook of Software Reliability Engineering*. McGraw-Hill, New York, New York, United States.

Moreira, A. , Araujo, J. , & Brito I. (2002). Crosscutting quality attributes for requirements engineering. In 14th International Conference on Software Engineering and Knowledge Engineering 2002 (pp. 167–174). Ischia, Italy.

Mylopoulos, J. , Chung, L. , & Nixon, B. (1992). Representing and using nonfunctional requirements: A process oriented approach. *IEEE Transactions in Software Engineering*, 18(6): 483–497.

Ncube, C. (2000). *A Requirements Engineering Method for COTS-Based Systems Development*, Ph.D. Thesis, City University, London.

O'Neil, F. (2015, July). Target data breach: Applying user-centered design principles to data breach notifications. In *Proceedings of the 33rd Annual International Conference on the Design of Communication* (pp. 1–8). Limerick, Ireland.

Park, D. , Kang, S. , & Lee, J. (2004). Design phase analysis of software performance using aspect oriented programming. In *5th Aspect-Oriented Modeling Workshop in Conjunction with UML 2004*. Lisbon, Portugal.

Robertson, S. , & Robertson, J. (1999). *Mastering the Requirements Process*. Addison-Wesley Professional, Boston, Massachusetts, United States.

Seffah, A. , Desmarais, M. , & Metzger, M. (2005). *Human-Centered Software Engineering*. Springer, Dordrecht, The Netherlands.

Siemens Warns of Possible Hearing Damage in Some Cell Phones . (2004). http://www.consumeraffairs.com/news04/siemens_mobile.html, last visited on 04 August 2009 .

Standard Glossary of Software Engineering Terminology . (1990). IEEE Standard 610.12-1990. DOI: 10.1109/IEEESTD.1990.101064.

Supakkul, S. , & Chung, L. (2004). Integrating FRs and NFRs: A use case and goal driven approach. In *Proceedings of the 2nd International Conference on Software Engineering Research, Management and Applications (SERA)* (pp. 30–37). Los Angeles, CA.

Weber, M. , & Wesbrot, J. (2003). Requirements engineering in automotive development: Experiences and challenges. *IEEE Software*, 20(1): 16–24.

Wieggers, K. (2003). *Software Requirements*, 2nd edition. Microsoft Press, Washington, USA.

Wikipedia: Non-Functional Requirements. http://en.wikipedia.org/wiki/Non-functional_requirements (visited May 2021).

Wikipedia: Requirements Analysis . http://en.wikipedia.org/wiki/Requirements_analysis (visited 05 July 2007).

Requirements Validations and Verifications

Aizenbud-Reshef, N. , Nolan, B. T. , Rubin, J. , & Shaham-Gafni, Y. (2006). Model traceability. *IBM System Journal*, 45(3): 515–526.

Arora, C. , Sabetzadeh, M. , Briand, L. , & Zimmer, F. (2015). Automated checking of conformance to requirements templates using natural language processing. *IEEE Transactions on Software Engineering*, 41(10): 944–996.

Bahill, T. A. , & Henderson, S. J. (2005). Requirements development, verification, and validation exhibited in famous failures. *Systems Engineering*, 8(1): 1–14.

Basili, V. , Caldiera, G. , & Rombach, H. D. (1994). Goal question metric approach. In J. J. Marciniak (Ed.), *Encyclopedia of Software Engineering* (pp. 528–532). Wiley, New York.

Boehm, B. W. (1984). Verifying and validating software requirements and design specifications. *IEEE Software*, 1(1): 75–88.

Carlson, N. , & Laplante, P. (2014). The NASA automated requirements measurement tool: A reconstruction. *Innovations in Systems and Software Engineering*, 10(2): 77–91.

Chung, L. , Nixon, B. A. , & Yu, E. (1995). Using non-functional requirements to systematically support change. *Proceedings of the Second IEEE International Symposium on Requirements Engineering* (pp. 132–139). York.

Cleland-Huang, J. (2005). Toward improved traceability of non-functional requirements. *Proceedings of the 3rd International Workshop on Traceability in Emerging Forms of Software Engineering* (pp. 14–19). Long Beach, CA.

Fagan, M. E. (1986). Advances in software inspections. *IEEE Transactions on Software Engineering*, 12(7): 744–751.

Femmer, H. , Fernández, D. M. , Juergens, E. , Klose, M. , Zimmer, I. , & Zimmer, J. (2014, June). Rapid requirements checks with requirements smells: Two case studies. In *Proceedings of the 1st International Workshop on Rapid Continuous Software Engineering* (pp. 10–19). ACM, Hyderabad, India.

Gervasi, V. , & Nuseibeh, B. (2002). Lightweight validation of natural language requirements. *Software: Practice and Experience*, 32(2): 113–133.

Gilb, T. (2005). *Competitive Engineering: A Handbook for Systems Engineering, Requirements Engineering, and Software Engineering Using Planguage*. Butterworth-Heinemann, Oxford.

Gotel, O. C. Z. (1995). *Contribution Structures for Requirements Traceability*, Doctoral dissertation, University of London, London, UK..

Hammer, T. F. , Huffman, L. L. , Rosenberg, L. H. , Wilson, W. , & Hyatt, L. (1998). Doing requirements right the first time. *CROSSTALK The Journal of Defense Software Engineering*, 1: 20–25.

Hetzel, B. (1988). *The Complete Guide to Software Testing*, 2nd edition. QED Information Sciences, Inc., London.

IEEE Std 1012-2012, IEEE Standard for Software Verification and Validation. (2004). Institute for Electrical and Electronics Engineers, Piscataway, NJ.

ISO/IEC/IEEE Std 29148, Systems and Software Engineering—Life Cycle Processes—Requirements Engineering. (2011). IEEE, Piscataway, NJ.

Kassab, M. , & Laplante, P. (2022). The current and evolving landscape of requirements engineering state of practice. *IEEE Software*. DOI: 10.1109/MS.2022.3147692.

Kassab, M. , Ormandjieva, O. , & Daneva, M. (2008). A traceability metamodel for change management of non-functional requirements. In *2008 Sixth International Conference on Software Engineering Research, Management and Applications* (pp. 245–254). IEEE, Prague, Czech Republic.

Lami, G. (2005). QuARS: A Tool for Analyzing Requirement (CMU/SEI-2005-TR-014). Software Engineering Institute, Carnegie Mellon University. <http://resources.sei.cmu.edu/library/asset-view.cfm?AssetID=7681> (accessed 7 January 2017).

Laplante, P. A. (2010). Stakeholder analysis for smart grid systems. In *IEEE Reliability Society Annual Technical Report*. <http://rs.ieee.org/images/files/Publications/2010/2010-02.pdf> (accessed June 2017).

Laplante, P. A. , Agresti, W. W. , & Djavanshir, G. R. (2007). Guest editor's introduction, special section on IT quality enhancement and process improvement. *IT Professional*, 9: 10–11.

Menzel, I. , Mueller, M. , Gross, A. , & Doerr, J. (2010). An experimental comparison regarding the completeness of functional requirements specifications. In *2010 18th IEEE International Requirements Engineering Conference (RE)* (pp. 15–24). IEEE, Sydney, NSW.

NASA (National Aeronautics and Space Administration) . (2008). Assurance Process for Complex Electronics. Updated 28 January 2008 . https://www.hq.nasa.gov/office/codeq/software/ComplexElectronics/I_requirements.htm (accessed January 2016).

O'Neil, R. (2004). Pennsylvania's "No Jake Braking" Signs. OLR Research Report #004-R-0515. <http://www.cga.ct.gov/2004/rpt/2004-R-0515.htm> (accessed March 2013).

Voas, J. , & Laplante, P. (2010). End brake retarder prohibitions: Defining "shall not" requirements effectively. *Computer*, 12(3): 46–53.

Wilson, W. M. , Rosenberg, L. H. , & Hyatt, L. E. (1997). Automated analysis of requirement specifications. In *Proceedings of the 19th International Conference on Software Engineering* (pp. 161–171). Boston, Massachusetts, USA.

Formal Methods

Abrial, J. R. (2009). Faultless systems: Yes we can! *Computer*, 42(9): 30–36.

Agrafiotis, I. , Creese, S. , & Goldsmith, M. (2011, September). Developing a strategy for automated privacy testing suites. In *IFIP PrimeLife International Summer School on Privacy and Identity Management for Life* (pp. 32–44). Springer, Berlin, Heidelberg.

Awodey, S. (2005). *Category Theory*. Carnegie Mellon University, Pittsburg, PA.

Böhm, C. , & Jacopini, G. (1966). Flow diagrams, Turing machines, and languages with only two formation rules. *Communications of the ACM*, 9(5): 366–371.

Bowen, J. P. , & Hinchey, M. G. (1995a). Seven more myths of formal methods. *Software*, 12(4): 34–41.

Bowen, J. P. , & Hinchey, M. G. (1995b). Ten commandments of formal methods. *Computer*, 28(4): 56–63.

Bowen, J. P. , Hinchey, M. G. , & Vassev, E. (2010). Formal requirements specification. In P. Laplante (Ed.), *Encyclopedia of Software Engineering* (pp. 321–332). Taylor & Francis, Boca Raton, FL.

Bruel, J. M. , Ebersold, S. , Galinier, F. , Mazzara, M. , Naumchev, A. , & Meyer, B. (2021). The role of formalism in system requirements. *ACM Computing Surveys (CSUR)*, 54(5): 1–36.

Crow, J. , & Di Vita, B. (1998). Formalizing space shuttle software requirements: Four case studies. *ACM Transactions on Software Engineering and Methodology*, 7(3): 296–332.

Fabian, B. , Gürses, S. , Heisel, M. , Santen, T. , & Schmidt, H. (2010). A comparison of security requirements engineering methods. *Requirements Engineering*, 15(1): 7–40.

Forster, T. E. (2003). *Logic, Induction and Sets*. Cambridge University Press, Cambridge.

Graves, H. , & Bijan, Y. (2011). Using formal methods with SysML in aerospace design and engineering. *Annals of Mathematics and Artificial Intelligence*, 63(1): 53–102.

Gries, D. , & Gries, P. (2005). *Multimedia Introduction to Programming Using Java*. Springer, New York.

Hacken, G. (2007). Private email communication to P. Laplante.

Hall, A. (1990). Seven myths of formal methods. *Software*, 7(5): 11–19.

Heitmeyer, C. (2007). Formal methods for specifying, validating, and verifying requirements. *Journal of Universal Computer Science*, 13(5): 607–618.

Hoare, C. A. R. (1969). An axiomatic basis for computer programming. *Communications of the ACM*, 12(10): 576–583.

Jang, D. , Tatlock, Z. , & Lerner, S. (2012). Establishing browser security guarantees through formal shim verification. In *The 21st USENIX Security Symposium (USENIX Security 12)* (pp. 113–128). Bellevue, WA.

Lano, K. (1996). *The B Language and Method*. Springer-Verlag, London, pp. 94–98.

Laplante, P. A. (2005). *Dictionary of Computer Science, Engineering and Technology*, 2nd edition. CRC Press, Boca Raton, FL.

Moy, Y. , Lednot, E. , Delseny, H. , Wiels, V. , & Monate, B. (2013). Testing or formal verification: Do-178c alternatives and industrial experience. *IEEE Software*, 30(3): 50–57.

NASA . (2012). *National Space Science Data Center: Mariner 1*.
<http://nssdc.gsfc.nasa.gov/nmc/masterCatalog.do?sc=MARIN1> (accessed March 2017).

Nuseibeh, B. (1997). Ariane 5: Who dunnit? *Software*, 14(3): 15–16.

Parnas, D. L. (2010). Really rethinking “formal methods.” *Computer*, 43(1): 28–34.

Richards, A. (2006). *Formalization of an EMS System Using Category Theory*, Master’s Thesis, Penn State University, State College, PA, under the supervision of P. Laplante.

Shehata, M. , & Eberlein, A. (2010). Requirements interaction detection. In P. Laplante (Ed.), *Encyclopedia of Software Engineering* (pp. 979–998). Taylor & Francis, New York.

Truss, L. (2003). *Eats Shoots and Leaves: The Zero Tolerance Approach to Punctuation*. Profile Books, London.

Requirements Specification and Agile Methodologies

Ambler, S. (2007). *Agile Requirements Change Management*.
<http://www.agilemodeling.com/essays/changeManagement.htm> (accessed January 2017).

Anthopoulos, L. , Reddickb, C. G. , & Giannakidou, I. (2016). Why e-government projects fail? An analysis of the Healthcare. *Gov. Government Information Quarterly*, 33(1): 161–173.

Bass, L. , Weber, I. , & Zhu, L. (2015). *DevOps: A Software Architect’s Perspective*. Addison-Wesley Professional, Boston, MA, USA.

Beck, K. (2000). *Extreme Programming Explained: Embrace Change*. Longman Higher Education, London.

Beck, K. , Beedle, M. , van Bennekum, A. , Cockburn, A. , Cunningham, W. , Fowler, M. , Grenning, J. , Highsmith, J. , Hunt, A. , Jeffries, R. and Kern, J. (2001). “Agile Manifesto” and “Principles Behind the Agile Manifesto.” <http://agilemanifesto.org/> (accessed January 2017).

Boehm, B. , and Turner, R. (2003). *Balancing Agility and Discipline: A Guide to the Perplexed*. Addison-Wesley, Boston, MA.

Bose, S. , Kurhekar, M. , & Ghoshal, J. (2008). *Agile Methodology in Requirements Engineering*. SETLabs Briefings Online, pp. 13–21.

Cao, L. , & Ramesh, B. (2008). Agile requirements engineering practices: An empirical study. *Software*, 25(1): 60–67.

Cleland-Huang, J. (2014). Don’t fire the architect! Where were the requirements? *IEEE Software*, 31(2): 27–29.

Cunningham, W. (1992). The WyCash portfolio management system. *ACM SIGPLAN OOPS Messenger*, 4(2): 29–30.

Inayat, I. , Moraes, L. , Daneva, M. , & Salim, S. S. (2015a, May). A reflection on agile requirements engineering: Solutions brought and challenges posed. In *Scientific Workshop Proceedings of the XP2015* (p. 6). ACM, Chicago, IL.

Inayat, I. , Salim, S. S. , Marczak, S. , Daneva, M. , & Shamshirband, S. (2015b). A systematic literature review on agile requirements engineering practices and challenges. *Computers in Human Behavior*, 51: 915–929.

Kassab, M. , & Laplante, P. (2022). The current and evolving landscape of requirements engineering state of practice. *IEEE Software*. DOI: 10.1109/MS.2022.3147692.

Kassab, M. , Neill, C. , & Laplante, P. (2014). State of practice in requirements engineering: Contemporary data. *Innovations in Systems and Software Engineering*, 10(4): 235–241.

Laplante, P. A. (2006). *What Every Engineer Needs to Know about Software Engineering*. CRC/Taylor & Francis, Boca Raton, FL.

Mugridge, R. (2008). Managing agile project requirements with story test-driven development. *Software*, 25(1): 68–75.

Orr, K. (2004). Agile requirements: Opportunity or oxymoron? *IEEE Software*, 21(3), 71–73.

Paetsch, F. , Eberlein, A. , & Maurer, F. (2003, June). Requirements engineering and agile software development. In *WET ICE 2003. Proceedings. Twelfth IEEE International Workshops on Enabling Technologies: Infrastructure for Collaborative Enterprises*, 2003 (pp. 308–313). IEEE, Linz, Austria.

Ramesh, B. , Cao, L. , & Baskerville, R. (2010). Agile requirements engineering practices and challenges: An empirical study. *Information Systems Journal*, 20(5): 449–480.

Rubin, E. , & Rubin, H. (2011). Supporting agile software development through active documentation. *Requirements Engineering*, 16(2): 117–132.

Savolainen, J. , Kuusela, J. , & Vilavaara, A. (2010, September). Transition to agile development—rediscovery of important requirements engineering practices. In *2010 18th IEEE International Requirements Engineering Conference* (pp. 289–294). IEEE, Sydney, NSW.

Schwaber, K. , & Beedle, M. (2001). *Agile Software Development with SCRUM*. Prentice Hall, Upper Saddle River, NJ.

Sillitti, A. , & Succi, G. (2006). Requirements engineering for agile methods. In A. Aurum & C. Wohlin (Eds.), *Engineering and Managing Software Requirements* (pp. 309–326). Springer, New York.

Venkatesh, V. , Hoehle, H. , & Aljafari, R. (2014). A usability evaluation of the Obamacare website. *Government Information Quarterly*, 31(4): 669–680.

Vlaanderen, K. , Jansen, S. , Brinkkemper, S. , & Jaspers, E. (2011). The agile requirements refinery: Applying SCRUM principles to software product management. *Information and Software Technology*, 53(1): 58–70.

Wagner, S. , Méndez-Fernández, D. , Kalinowski, M. , & Felderer, M. (2018). Agile requirements engineering in practice: Status quo and critical problems. *CLEI Electronic Journal*, 21(1): 15.

West, D. , Grant, T. , Gerush, M. , & D'Silva, D. (2010). *Agile Development: Mainstream Adoption Has Changed Agility*. Forrester Research, Cambridge, MA.

Williams, L. (2004). *Agile Requirements Elicitation*. <http://agile.csc.ncsu.edu/SEMaterials/AgileRE.pdf> (accessed January 2017).

Tool Support for Requirements Engineering

Carrillo de Gea, J. M. , Nicolás, J. , Alemán, J. L. F. , Toval, A. , Ebert, C. , & Vizcaíno, A. (2011). Requirements engineering tools. *IEEE Software*, 28(4): 86–91.

Carrillo de Gea, J. M. , Nicolás, J. , Fernández, A. J. L. , Toval, A. , Ebert, C & Vizcaíno, A. (2015). Commonalities and differences between requirements engineering tools: A quantitative approach. *Computer Science and Information Systems*, 12(1): 257–288.

Carrillo de Gea, J. M. , Ebert, C. , Hosni, M. , Vizcaíno, A. , Nicolás, J. , & Fernández-Alemán, J. L. (2021). Requirements engineering tools: An evaluation. *IEEE Software*, 38(3): 17–24.

Cleland-Huang, J. , Settimi, R. , Romanova, E. , Berenbach, B. , & Clark, S. (2007). Best practices for automated traceability. *IEEE Computer*, 40: 27–35.

Costello, R. J. , & Liu, D. B. (1995). Metrics for requirements engineering. *Journal of Systems and Software*, 29(1): 39–63.

Dalpiaz, F. , & Niu, N. (2020). Requirements engineering in the days of artificial intelligence. *IEEE Software*, 37(4): 7–10.

Daud, N. , Kamalrudin, M. , Sidek, S. , & Ahmad, S. S. S. (2014). A review of requirements engineering tools for requirements validation. *International Journal of Software Engineering and Technology*, 1(1).

De Lucia, A. , & Qusef, A. (2010). Requirements engineering in agile software development. *Journal of Emerging Technologies in Web Intelligence*, 2(3): 212–220.

Eriksson, M. , Morasi, H. , Borstler, J. , & Borg, K. (2005). The PLUSS toolkit—Extending telelogic DOORS and IBM-rational rose to support product line in use case modeling. In *Proceedings of the 20th IEEE/ACM International Conference on Automated Software Engineering* (pp. 300–304). Long Beach, CA.

Ferreira, D. , & da Silva, A. R. (2008, September). Wiki supported collaborative requirements engineering. In *Proceedings of the 4th International Symposium on Wikis*. ACM, Porto, Portugal.

FitNesse project . (2007). www.fitnessse.org (accessed January 2017).

FreeMind project . (2017). http://freemind.sourceforge.net/wiki/index.php/Main_Page (accessed February 2017).

Gandhi, P. , Haugen, N. C. , Hill, M. , & Watt, R. (2005). Creating a living specification using fit documents. In Agile 2005 Conference (Agile 05) (pp. 253–258). IEEE CS Press, Denver, CO.

Hammer, T. F. , Huffman, L. L. , Rosenberg, L. H. , Wilson, W. , & Hyatt, L. (1998). Doing requirements right the first time. CROSSTALK The Journal of Defense Software Engineering, 20–25.

Heindl, M. , Reinish, F. , Biffi, S. , & Egyed, A. (2006, August). Value-based selection of requirements engineering tool support. In 32nd EUROMICRO Conference on Software Engineering and Advanced Applications, SEAA' 06 (pp. 266–273), Cavtat, Croatia.

Hull, E. , Jackson, K. , & Dick, J. (2011). Doors: A tool to manage requirements. Requirements Engineering, 181–198. Springer-Verlag, London.

ISO/IEC TR 24766 . (2009). Information Technology—Systems and Software Engineering—Guide for Requirements Engineering Tool Capabilities. International Organization for Standardization (ISO), Geneva, Switzerland.

Kassab, M. (2014). An empirical study on the requirements engineering practices for agile software development. In 2014 40th EUROMICRO Conference on Software Engineering and Advanced Applications (SEAA). IEEE.

Kassab, M. , & Laplante, P. (2021). The current and evolving landscape of requirements engineering state of practice. IEEE Software. DOI Bookmark: 10.1109/MS.2022.3147692

Kassab, M. , Ormandjieva, O. , & Daneva, M. (2008). A traceability metamodel for change management of non-functional requirements. In 2008 Sixth International Conference on Software Engineering Research, Management and Applications (pp. 245–254). IEEE, Prague, Czech Republic.

Kassab, M. , Ormandjieva, O. , & Daneva, M. (2009). An ontology based approach to non-functional requirements conceptualization. In 2009 Fourth International Conference on Software Engineering Advances (pp. 299–308). IEEE, Porto, Portugal.

Krippendorff, K. (2004). Content Analysis: An Introduction to Its Methodology. Sage, Thousand Oaks, CA.

Kurtz, G. , Geisser, M. , Hildenbrand, T. , & Kude, T. (2008). Mobile technologies in requirements engineering. In Advances in Computer and Information Sciences and Engineering (pp. 317–322). Springer, Dordrecht.

Lee, S.-W. , Gandhi, R. A. , & Park, S. (2011). Requirement tracing. In P. Laplante (Ed.), Encyclopedia of Software Engineering (pp. 999–1011). Taylor & Francis, Boca Raton, FL.

Li, J. , Zhu, L. , Jeffery, R. , Liu, Y. , Zhang, H. , Wang, Q. , & Li, M. (2012). An initial evaluation of requirements dependency types in change propagation analysis.

Li, T. , Zhang, F. , & Wang, D. (2018). Automatic user preferences elicitation: A data-driven approach. In International Working Conference on Requirements Engineering: Foundation for Software Quality (pp. 324–331). Springer, Cham, Utrecht, The Netherlands.

Liang, P. , Avgeriou, P. , & Clerc, V. (2009, July). Requirements reasoning for distributed requirements analysis using semantic wiki. In Fourth IEEE International Conference on Global Software Engineering, 2009. ICGSE 2009 (pp. 388–393). IEEE, Limerick, Ireland.

Lutz, R. , Schäfer, S. , & Diehl, S. (2012, September). Using mobile devices for collaborative requirements engineering. In Proceedings of the 27th IEEE/ACM International Conference on Automated Software Engineering (pp. 298–301). ACM, Essen, Germany.

Mäder, P. , Jones, P. L. , Zhang, Y. , & Cleland-Huang, J. (2013). Strategic traceability for safety-critical projects. IEEE Software, 30(3): 58–66.

Maiden, N. , Omo, O. , Seyff, N. , Grunbacher, P. , & Mitteregger, K. (2007). Determining stakeholder needs in the workplace: How mobile technologies can help. IEEE Software, 24: 46–52.

Martakis, A. , & Daneva, M. (2013). Handling requirements dependencies in agile projects: A focus group with agile software development practitioners. In IEEE 7th International Conference on Research Challenges in Information Science (RCIS) (pp. 1–11). IEEE, Paris, France.

RTCA. (2011). Software Considerations in Airborne Systems and Equipment Certification. DO-178C. Radio Technical Commission for Aeronautics (RTCA), Washington, DC.

Russell, S. , & Creighton, O. (2011, August). Virtual world tools for requirements engineering. In 2011 Fourth International Workshop on Multimedia and Enjoyable Requirements Engineering-Beyond Mere Descriptions and with More Fun and Games (MERE) (pp. 17–20). Trento, Italy.

Sharma, S. , & Pandey, S. K. (2013). Revisiting requirements elicitation techniques. International Journal of Computer Applications, 75(12): 35–39.

Sharma, S. , & Pandey, S. K. (2014). Integrating AI techniques in SDLC: Requirements phase perspective. International Journal of Computer Applications in Technology, 5(3): 1362–1436.

Spoletini, P. , & Ferrari, A. (2017). Requirements elicitation: A look at the future through the lenses of the past. In 2017 IEEE 25th International Requirements Engineering Conference (RE) (pp. 476–477). IEEE, Lisbon, Portugal.

Sud, R. R. , & Arthur, J. D. (2003). Requirements Management Tools: A Quantitative Assessment. Technical Report TR-03-10. Computer Science, Virginia Tech, Blacksburg, VA.

Volere requirements resources . (2013). <http://www.volere.co.uk/tools.htm> (accessed February 2017).

Yang, D. , Wu, D. , Koolmanojwong, S. , Brown, A. W. , & Boehm, B. W. (2008, January). WikiWinWin: A wiki based system for collaborative requirements negotiation. In Proceedings of the 41st Annual Hawaii International Conference on System Sciences (pp. 24–24). IEEE, Waikoloa, Big Island, Hawaii.

Yue, T. , Briand, L. C. , & Labiche, Y. (2011). A systematic review of transformation approaches between user requirements and analysis models. *Requirements Engineering*, 16(2): 75–99.

Requirements Management

Alfonsi, B. (2005). FBI's virtual case file living in limbo. *Security & Privacy*, 3(2): 26–31.

Andriole, S. (1998). The politics of requirements management. *IEEE Software*, 15: 82–84.

Berander, P. , Damm, L. O. , Eriksson, J. , Gorschek, T. , Henningsson, K. , Jönsson, P. , Kågström, S. , Milicic, D. , Mårtensson, F. , Rönkkö, K. , Tomaszewski, P. (2005). Software quality attributes and trade-offs. *Blekinge Institute of Technology*, 97(98): 19.

Bhat, J. M. , Gupta, M. , & Murthy, S. N. (2006). Overcoming requirements engineering challenges: Lessons from offshore outsourcing. *IEEE Software*, 23: 38–44.

Brown, W. J. , Malveau, R. C. , McCormick, H. W. , and Mowbray, T. J. (1998). *AntiPatterns: Refactoring Software, Architectures, and Projects in Crisis*. Wiley, New York.

Dekkers, C. A. , & McQuaid, P. A. (2002). The dangers of using software metrics to (mis) manage. *IT Professional*, 4(2): 24–30.

Ebert, C. (2010). Requirements engineering: Management. In P. Laplante (Ed.), *Encyclopedia of Software Engineering* (pp. 932–948). Taylor & Francis, Boca Raton, FL.

Goldstein, H. (2005). Who killed the virtual case file? *Spectrum*, 42(9): 24–35.

Hull E. , Jackson K. , Dick J. (2002). Management Aspects of Requirements Engineering. In: *Requirements Engineering. Practitioner Series*. Springer, London. https://doi.org/10.1007/978-1-4471-3730-6_8.

ISO 9000 Quality Management Systems . (2015). <http://www.iso.org> (accessed January 2017).

Israel, J. W. (2012). Why the FBI can't build a case management system. *Computer*, 45(6): 73–80.

Kassab, M. (2013, May). An integrated approach of AHP and NFRs framework. In *IEEE 7th International Conference on Research Challenges in Information Science (RCIS)* (pp. 1–8). IEEE, Paris, France.

Kassab, M. (2014, June). Early effort estimation for quality requirements by AHP. In *14th International Conference on Computational Science and Its Applications* (pp. 106–118). Springer International Publishing, Guimaraes, Portugal.

Kassab, M. , & Kilicay-Ergin, N. (2015). Applying analytical hierarchy process to system quality requirements prioritization. *Innovations in Systems and Software Engineering*, 11(4): 303–312.

Laplante, P. A. (2006). *What Every Engineer Needs to Know about Software Engineering*. CRC/Taylor & Francis, Boca Raton, Florida, United States.

Lawrence, B. (1996). Unresolved ambiguity. *American Programmer*, 9(5): 17–22.

Mahaux, M. , & Maiden, N. (2008). Theater improvisers know the requirements game. *Software*, 25: 68–69.

Neill, C. J. , Laplante, P. A. , & DeFranco, J. (2012). *Antipatterns: Identification, Refactoring, and Management*, 2nd edition. CRC Press, Boca Raton, FL.

Norden, B. (2007). Screenwriting for requirements engineers. *Software*, 24: 26–27.

Paulk, M. , Weber, C. V. , Garcia, S. M. , Chrissis, M.-B. C. , & Bush, M. (1993). *CMM Practices Manual*. CMU/SEI-93-TR-25. <https://www.sei.cmu.edu/reports/93tr024.pdf> (accessed February 2017).

Reinhart G. , Meis J. (2012). Requirements Management as a Success Factor for Simultaneous Engineering. In: ElMaraghy H. (eds). *Enabling Manufacturing Competitiveness and Economic Sustainability*. Springer, Berlin, Heidelberg. https://doi.org/10.1007/978-3-642-23860-4_36

Saaty T.L. (1988). What is the Analytic Hierarchy Process?. In: Mitra G. , Greenberg H.J. , Lootsma F.A. , Rijklaert M.J. , Zimmermann H.J. (eds). *Mathematical Models for Decision Support*. NATO ASI Series (Series F: Computer and Systems Sciences), vol 48. Springer, Berlin, Heidelberg. https://doi.org/10.1007/978-3-642-83555-1_5

Schmid K. (2014). Challenges and Solutions in Global Requirements Engineering – A Literature Survey. In: Winkler D. , Biffi S. , Bergsmann J. (eds) *Software Quality. Model-Based Approaches for Advanced Software and Systems Engineering*. SWQD 2014. *Lecture Notes in Business Information Processing*, vol 166. Springer, Cham. https://doi.org/10.1007/978-3-319-03602-1_6

Sinha, V. , & Sengupta, B. (2006). Enabling collaboration in distributed requirements management. *Software*, 23: 52–61.

Wieggers, Karl E. (2003). *Software Requirements*, 2nd edition. Microsoft Press, Redmond, Washington, United States.

Value Engineering of Requirements

- Abran, A. (2020). IEEE 2430 non-functional sizing measurements: A numerical placebo. *IEEE Software*, 38(3): 113–120.
- Albrecht, J. (1979, October). Measuring application development productivity. In *Proceedings of the IBM Applications Development Symposium* (pp. 14–17). Monterey, CA.
- Angelis, L. , Stamelos, I. , & Morisio, M. (2001). Building a software cost estimation model based on categorical data. In *Proceedings of the 7th International Symposium on Software Metrics* (pp. 4–15) METRICS. IEEE Computer Society, Washington, DC.
- Boehm, B. W. (1981). *Software Engineering Economics*. Prentice Hall, Englewood Cliffs, NJ.
- Clemmons, R. K. (2006). Project estimation with use case points. *The Journal of Defense Software Engineering*, 2: 18–22.
- ISO/IEC 14143-6:2012-Functional Size Measurement . (2012). <https://www.iso.org/standard/60176.html>
- ISO/IEC 19761:2011 Software Engineering — COSMIC: A Functional Size Measurement Method . (2011). <https://www.iso.org/standard/54849.html>
- Jones, C. (1996). Activity-based software costing. *IEEE Computer*, 29: 103–104.
- Kassab, M. , & Laplante, P. (2022). The current and evolving landscape of requirements engineering state of practice. *IEEE Software*. Accepted.
- Kassab, M. , Neill, C. , & Laplante, P. (2014). State of practice in requirements engineering: Contemporary data. *Innovations in Systems and Software Engineering*, 10(4): 235–241.
- Kassab, M. , Ormandjieva, O. , Daneva, M. , & Abran, A. (2007). Non-functional requirements size measurement method (NFSM) with COSMIC-FFP. In *Software Process and Product Measurement* (pp. 168–182). Springer, Berlin, Heidelberg.
- Liebchen, G. A. , & Shepperd, M. (2005). Software productivity analysis of a large data set and issues of confidentiality and data quality. In *Proceedings of the 11th IEEE International Software Metrics Symposium* (September 19–22, 2005, p. 46). METRICS, IEEE Computer Society, Washington, DC.
- Liu, K. , Valerdi, R. , & Laplante, P. A. (2010). Better requirements decomposition guidelines can improve cost estimation of systems engineering and human systems integration. In *8th Annual Conference on Systems Engineering Research*. Hoboken, NJ.
- Magazinovic, A. , & Pernstål, J. (2008). Any other cost estimation inhibitors? In *Proceedings of the Second ACM-IEEE International Symposium on Empirical Software Engineering and Measurement* (pp. 233–242), Kaiserslautern, Germany, ESEM '08. ACM, New York.
- Maxwell, K. D. , & Forselius, P. (2000). Benchmarking software development productivity. *IEEE Software*, 17(1): 80–88.
- McConnell, S. (1997). *Software Project Survival Guide*. Microsoft Press, Redmond, WA.
- Morgan, J. N. (2005). A roadmap of financial measures for IT project ROI. *IT Professional*, 7: 52–57.
- Nassif, A. B. , Capretz, L. F. , & Ho, D. (2012a, December). Estimating software effort using an ANN model based on use case points. In *2012 11th International Conference on Machine Learning and Applications (ICMLA)* (Vol. 2, pp. 42–47). IEEE, Boca Raton, Florida, USA.
- Nassif, A. B. , Capretz, L. F. , Ho, D. , & Azzeh, M. (2012b, December). A Treeboost model for software effort estimation based on use case points. In *2012 11th International Conference on Machine Learning and Applications (ICMLA)* (Vol. 2, pp. 314–319). IEEE, Boca Raton, Florida, USA.
- Park, R. E. (1992). *Software Size Measurement: A Framework for Counting Source Statements*. Carnegie-Mellon University, Software Engineering Institute, Pittsburgh, PA.
- Pressman, R. S. (2015). *Software Engineering: A Practitioner's Approach*. McGraw-Hill Higher Education. New York, USA.
- Raffo, D. , Settle, J. , & Harrison, W. (1999). Investigating Financial Measures for Planning of Software IV&V. *Research Report #TR-99-05*. Portland State University, Portland, OR.
- Reeves, J. , DePasquale, D. , & Lim, E. (2010, August). Affordability engineering: Bridging the gap between design and cost. In *AIAA SPACE 2010 Conference & Exposition* (p. 8904), Anaheim, California.
- Reifer, D. (2002). Estimating web development costs: There are differences. *Crosstalk*, 15(6): 13–17.
- Shane, J. S. , Molenaar, K. R. , Anderson, S. , & Schexnayder, C. (2009). Construction project cost escalation factors. *Journal of Management in Engineering*, 25(4): 221–229.
- Sigmund, P. (2007). Triumph, tragedy mark Boston's big dig project. In *Construction Equipment Guide*. <http://www.constructionequipmentguide.com/triumph-tragedy-mark-bostons-big-dig-project/8751> (accessed February 2017).
- Software Non-Functional Assessment Process (SNAP) , Assessment Practices Manual, Release 2.4. (2017). International Function Points Users Group, Princeton Junction, NJ.
- Trial-Use for Software Non-Functional Requirements Sizing Measurement . (2019). IEEE Standard 2430, IEEE Standards Association, Piscataway, NJ.
- Valerdi, R. (2008). *The Constructive Systems Engineering Cost Model (COSYSMO): Quantifying the Costs of Systems Engineering Effort in Complex Systems*. VDM Verlag, Berlin.

Requirements Engineering: A Road Map to the Future

- L. Mertz , "Dream It, Design It, Print It in 3-D: What Can 3-D Printing Do for You?," in IEEE Pulse, vol. 4, no. 6, pp. 15–21, Nov. 2013, doi: 10.1109/MPUL.2013.2279616.
- C. C. d. Santos , M. E. Delamaro and F. L. S. Nunes , "The Relationship between Requirements Engineering and Virtual Reality Systems: A Systematic Literature Review," 2013 XV Symposium on Virtual and Augmented Reality, 2013, pp. 53–62, doi: 10.1109/SVR.2013.52.
- Aman Bhimani , "Feasibility of Using Virtual Reality in Requirements Elicitation Process," Master's Thesis, Kennesaw State University, pp. 11–28, 2017, https://digitalcommons.kennesaw.edu/cgi/viewcontent.cgi?article=1001&context=msse_etd
- Azar, J. , Smith, R. K. , & Cordes, D. (2007). Value-oriented requirements prioritization in a small development organization. *IEEE Software*, 24(1): 32–37.
- Bose, S. , Kurhekar, M. , & Ghoshal, J. (2008). Agile methodology in requirements engineering. *SETLabs Briefings Online*, 13–21.
- Chakraborty, P. , Shahriyar, R. , Iqbal, A. , & Bosu, A. (2018, October). Understanding the software development practices of blockchain projects: A survey. In *Proceedings of the 12th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement* (pp. 1–10), Oulu, Finland.
- Chechik, M. (2019, September). Uncertain requirements, assurance and machine learning. In *2019 IEEE 27th International Requirements Engineering Conference (RE)* (pp. 2–3). IEEE, Jeju Island, South Korea.
- Dalpiaz, F. , & Niu, N. (2020). Requirements engineering in the days of artificial intelligence. *IEEE Software*, 37(4): 7–10.
- del Sagrado, J. , & del Aguila, I. M. (2018). Stability prediction of the software requirements specification. *Software Quality Journal*, 26(2): 585–605.
- Demi, S. (2020, August). Blockchain-oriented requirements engineering: A framework. In *2020 IEEE 28th International Requirements Engineering Conference (RE)* (pp. 428–433). IEEE, Zurich, Switzerland.
- Destefanis, G. , Marchesi, M. , Ortu, M. , Tonelli, R. , Bracciali, A. , & Hierons, R. (2018, March). Smart contracts vulnerabilities: A call for blockchain software engineering? In *2018 International Workshop on Blockchain Oriented Software Engineering (IWBOSE)* (pp. 19–25). IEEE, Gothenburg, Sweden.
- European Commission. (2005). *The New SME Definition: User Guide and Model Declaration*. Section. Office for Official Publications of the European Communities, Brussels.
- Ferrari, A. , & Esuli, A. (2019). An NLP approach for cross-domain ambiguity detection in requirements engineering. *Automated Software Engineering*, 26(3): 559–598.
- Georgescu, M. , & Popescu, D. (2015). How could Internet of Things change the E-learning environment. *The 11th International Scientific Conference eLearning and Software for Education*, Bucharest, Romania.
- Groen, E. C. , Seyff, N. , Ali, R. , Dalpiaz, F. , Doerr, J. , Guzman, E. , ... Stade, M. (2017). The crowd in requirements engineering: The landscape and challenges. *IEEE Software*, 34(2): 44–52.
- Gubbi, J. , Buyya, R. , Marusic, S. , & Palaniswami, M. (2013). Internet of Things (IoT): A vision, architectural elements, and future directions. *Future Generation Computer Systems*, 29(7): 1645–1660.
- Han, W. (2011). Research of intelligent campus system based on IOT. In *Advances in Multimedia, Software Engineering and Computing* (Vol. 1, pp. 165–169). Springer.
- Hayes, J. H. , Payne, J. , & Leppelmeier, M. (2019, September). Toward improved artificial intelligence in requirements engineering: Metadata for tracing datasets. In *2019 IEEE 27th International Requirements Engineering Conference Workshops (REW)* (pp. 256–262). IEEE, Jeju Island, South Korea.
- JPMorgan Chase Co . (2018). *Small Businesses Are an Anchor of the US Economy*. <https://www.jpmorganchase.com/institute/research/smallbusiness/small-business-dashboard/economic-activity>.
- Kandaswamy, R. , & Furlonger, D. (2018). Blockchain-based transformation: A Gartner trend insight report. <https://www.gartner.com/en/doc/3869696-blockchain-based-transformation-a-gartner-trend-insight-report>
- Kassab, M. (2015, August). The changing landscape of requirements engineering practices over the past decade. In *2015 IEEE Fifth International Workshop on Empirical Requirements Engineering (EmpiRE)* (pp. 1–8). IEEE, Ottawa, Canada.
- Kassab, M. (2021). How requirements engineering is performed in small businesses? Proceeding of the Workshop on Requirement Engineering for Software Startups and Emerging Technologies in Conjunction with 29th IEEE International Requirements Engineering Conference 2021. *Virtual*.
- Kassab, M. , & Laplante, P. (2022). The current and evolving landscape of requirements engineering state of practice. *IEEE Software*. DOI: 10.1109/MS.2022.3147692.
- Kassab, M. , Destefanis, G. , DeFranco, J. , & Pranav, P. (2021, May). Blockchain-Engineers wanted: An empirical analysis on required skills, education and experience. In *2021 IEEE/ACM 4th International Workshop on Emerging Trends in Software Engineering for Blockchain (WETSEB)* (pp. 49–55). IEEE.
- Laplante, P. A. , Kassab, M. , Laplante, N. L. , & Voas, J. M. (2017). Building caring healthcare systems in the Internet of Things. *IEEE Systems Journal*, 12(3): 3030–3037.
- Laplante, P. A. , Laplante, N. , & Voas, J. (2015). Considerations for healthcare applications in the Internet of Things. *Rel. Dig.*, 61(4): 8–9.
- Lenz, L. , Pomp, A. , Meisen, T. , & Jeschke, S. (2016). How will the Internet of Things and Big Data analytics impact the education of learning-disabled students? A concept paper. In *3rd MEC International Conference on*

Big Data and Smart City (ICBDSC) (pp. 1–7). IEEE, Muscat, Oman.

McCarthy, J. (2007). What Is Artificial Intelligence? Computer Science Department, Stanford University, Stanford, CA. <http://www-formal.stanford.edu.ezaccess.libraries.psu.edu/jmc/whatisai.pdf>

Mell, P. , & Grance, T. (2011). The NIST definition of cloud computing. <https://csrc.nist.gov/publications/detail/sp/800-145/final>

Peclat, R. N. , & Ramos, G. N. (2018). Semantic analysis for identifying security concerns in software procurement edicts. *New Generation Computing*, 36(1): 21–40.

Rashwan, A. (2012, May). Semantic analysis of functional and non-functional requirements in software requirements specifications. In *Canadian Conference on Artificial Intelligence* (pp. 388–391). Springer, Berlin, Heidelberg.

Stanik, C. , Haering, M. , & Maalej, W. (2019, September). Classifying multilingual user feedback using traditional machine learning and deep learning. In *2019 IEEE 27th International Requirements Engineering Conference Workshops (REW)* (pp. 220–226). IEEE, Jeju Island, South Korea.

Sula, A. , Spaho, E. , Matsuo, K. , Barolli, L. , Miho, R. , & Xhafa, F. (2013). An IoT-based system for supporting children with autism spectrum disorder. In *2013 Eighth International Conference on Broadband and Wireless Computing, Communication and Applications* (pp. 282–289), IEEE, Compiègne, France.

U.S. Small Business Administration . <https://www.sba.gov>

Uzelac, A. , Gligoric, N. , & Krco, S. (2015). A comprehensive study of parameters in physical environment that impact students' focus during lecture using Internet of Things. *Computers in Human Behavior*, 53: 427–434.

Voas, J. M. NIST SP 800-183 Networks of 'Things'. <http://dx.doi.org/10.6028/NIST.SP.800-183>, <http://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-183.pdf>, 2016

Wind, S. , & Schrödl, H. (2010, December). Requirements engineering for cloud computing: A comparison framework. In *International Conference on Web Information Systems Engineering* (pp. 404–415). Springer, Berlin, Heidelberg.